Numeriske Simuleringer av Bølgeforplantning i Betong

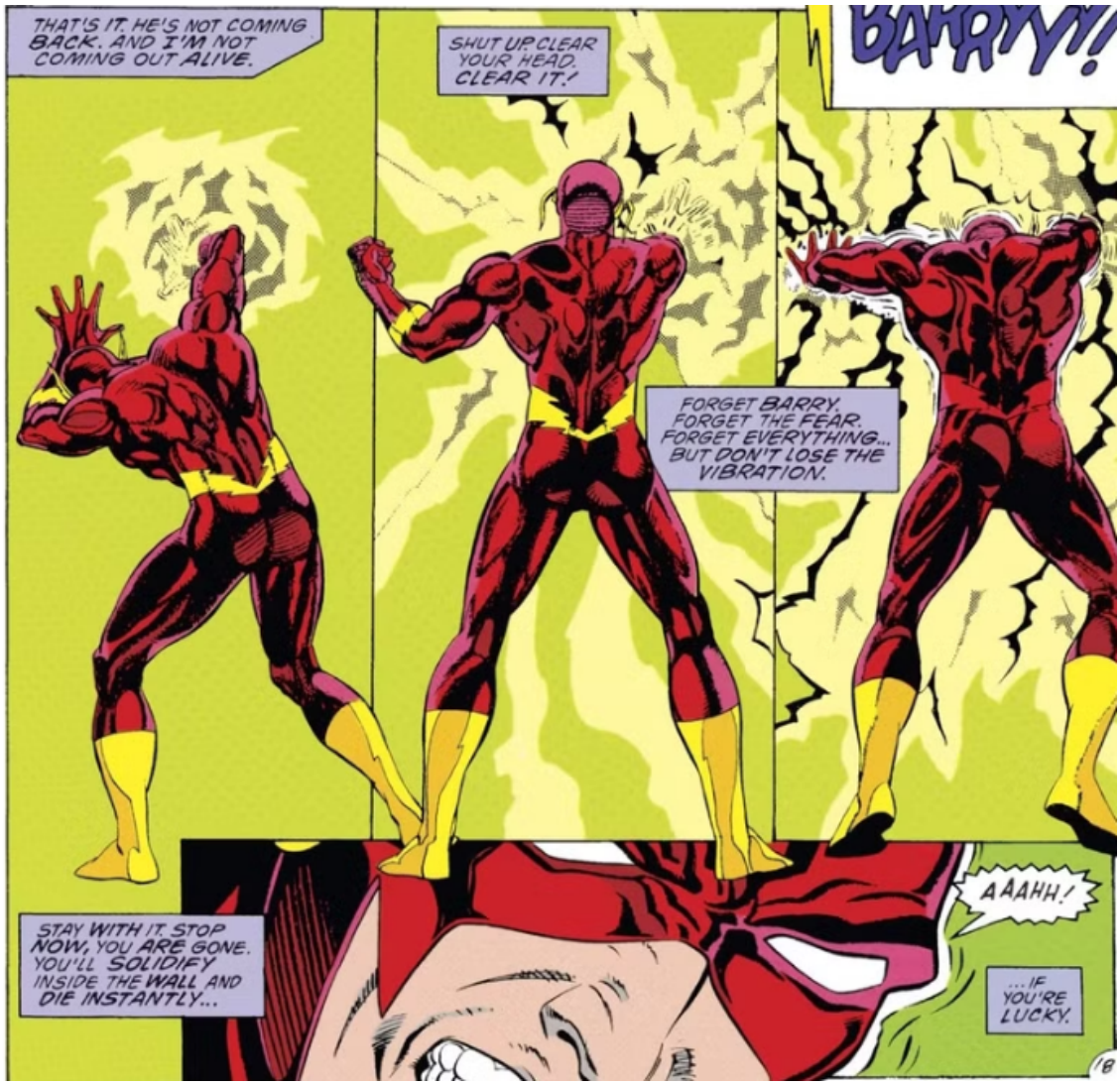
April 1, 2026

1 Hvor mye kraft må The Flash produsere for å sprekke betong?

Flere ganger i Flash-serien og tegnebøkene bruker superhelten The Flash hans evne til å vibrere kroppen hans i ekstrem hastighet for å gjennomføre bestemte handlinger. Eksempler på slike handlinger er å vibrere molekylene hans med en frekvens som lar han passere gjennom solid materie, skape kraftige sjokkbølger. Han kan gjøre seg selv usynlig og vibrere. Problemstillingen er interessant fordi den kan gi oss innblikk i teknologi ved bruk av vibrasjon. I denne artikkelen skal vi modellere The Flash sin evne til å knuse/sette sprekker i en vegg ved å vibrere kroppen hans. Mer spesifikt, leter vi etter hvor stor kraft The Flash må produsere for å sprekke betongen.

[1]:
```
import numpy as np
from wavespeed import qL
from wavesolver2 import boundary_conditions
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
from numpy import newaxis
from matplotlib.animation import FuncAnimation
from matplotlib import cm
from numba import jit, float64, int32, boolean
from numba.experimental import jitclass
from IPython.display import Image
from IPython.display import HTML
Image(filename='C://users//toosi//Pictures/Flash.png')
```

[1]:



2 Klasse for kjøring

Under er en klasse for kjøring av 2D simulasjonene. Koden er forklart i deler i løpet av essayet. Merk at dessverre så er noen av variablene i konstruktoren (dx, dy, Nt) egentlig unødvendige siden de ikke brukes til å definere meshen…

[ ]:
```
#Spesifiserer verdier som initialiseres i wavesolver (For numba)
spec = [
    ('dx', float64),
    ('dy', float64),
    ('Nx', int32),
    ('Ny', int32),
    ('Nt', int32),
    ('xn', float64),
```

```
    ('yn', float64),
    ('tn', float64),
    ('b', float64),
    ('F', float64),
    ('K', float64),
    ('constant', boolean),
    ('points', float64[:, :]),
    ('dpoints', float64[:]),
    ('c', float64)
]
@jitclass(spec)
class wavesolver:

    def __init__(self, b, F, dx, dy, Nx, Ny, Nt, xn, yn, tn, K=28e+9):
        """
        Args:
            b - dempingskoeffisient
            F - kraftmodulus
            dx - Spacing
            dy - Spacing
            Nx - Antall x punkter
            Ny - Antall y punkter
            Nt - Antall tidspunkter
            xn - x sluttpunkt
            yn - y sluttpunkt
            tn - tid sluttpunkt
            K - bulkmodul
        """
        self.b=b
        self.dx=dx; self.dy=dy
        self.Nx=Nx; self.Ny=Ny; self.Nt=Nt
        self.xn=xn; self.yn=yn;self.tn=tn
        self.F = F; self.K=K; self.constant = False
        self.c = 0.0; self.points=np.array([[0, 0.5], [1, 0.5], [0.5, 0.5], [0.
↪5, 0], [0.5, 1]], dtype=np.float64)
        self.dpoints=np.array([2350, 2250, 2400, 2200, 2225], dtype=np.float64)


    def set_mesh(self, dx, dy, Nx, Ny, Nt):
        """
        Funksjon/Metode for å manuelt sette mesh
        Args: dx, dy, Nx, Ny, Nt
        """
        self.dx = dx; self.dy=dy
        self.Nx=Nx; self.Ny=Ny; self.Nt=Nt

    def change_mesh(self, refine, coarse, factor=0.5):
```

```
        """
        Metode for å minske eller øke resolusjon på mesh
        Args:
            Refine (bool)
            Coarse (bool)
        """
        if refine==True:
            self.dx=self.dx*factor; self.dy=self.dy*factor
        if coarse==True:
            self.dx=self.dx*(1/factor); self.dy=self.dy*(1/factor)

    def I(self, x, y):
        """
        Vektorisert funksjon for initial betingelse (unødvendig?)
        Args:
            x (Array)
            y (Array)
        Returnerer:
        Broadcastet array shape x*y bestående av nuller
        """
        return np.zeros_like(x*y)

    def rho2(self, x, y):
        """
        Funksjon for varierende massetetthet
        Args:
            x, y - Koordinater til punkt
        Returnerer:
            Massetetthet i punkt
        """
        points = self.points
        dpoints = self.dpoints
        for i in range(points.shape[0]): #Looper over punktene og sjekke om de␣
↪er like
            point = points[i]
            if (np.abs(point[0] - x) < 1e-5 and
                np.abs(point[1] - y) < 1e-5):
                return dpoints[i]
        d = np.zeros(points.shape[0], dtype=np.float64) #Array for tetthet
        for k in range(points.shape[0]): #Loop over punktene og sjekke avstand
            dx = x - points[k, 0] #x-avstand
            dy = y - points[k, 1] #y-avstand
            d[k] = (dx**2 + dy**2 )**0.5 #Avstand
            if d[k] < 1e-5: #Toleranse for avstand
                    return dpoints[k]
        w = np.zeros(d.shape[0]) #Array for vekting
        for i in range(d.shape[0]): #Regner ut vekter
```

```
                w[i] = 1 / (d[i]**2)
        w_sum = np.sum(w) #Summen av vektene
        rho = np.sum(dpoints * w) / w_sum #Regner ut massetetthet
        if self.constant ==True:
            return 2400
        return rho


    def qL(self, x, y):
        """
        Funksjon for bølgefart:
        Args:
            x, y - Koordinater for punkt
        Returnerer:
            Bølgefart i punkt
        """

        rho = self.rho2; K = self.K
        qL = K/rho(x, y)
        if self.constant==True:
            return self.c**2
        return qL

    def change_speed(self, c, constant):
        """
        Metode for å sette konstant fart, konstant massetetthet og bølgefart:
        Args:
            c - bølgefart
            constant (bool) - Gir konstant massetetthet og bølgefart
        """
        constant = True
        self.c = c
        self.constant = constant

    def f(self, x, y, t, dt, wF=1500*2*np.pi):
        """
        Funksjon for kraft i punkt
        Args:
            x, y - Koordinater til punkt
            t - tid
            wF - vinkelfrekvens
        """
        F = self.F
        A = (0.75-0.25)*(0.3-0.1)
        f =-1/(A)*F*wF**2*np.cos(wF*t)
        return np.where((0.25 <= x) & (x <= 0.75) & (0.1 <= y) & (y <= 0.3), f,␣
↪0)
```

```python
def boundary_conditions_vectorized(self, p):
        """
        Grensebetingelser
        Args:
            p (array)
        Returnerer:
            p (array) med verdi 0 i ytre punkter
        """
        Nx, Ny =p.shape
        for i in range(Nx):
            p[i, 0] = 0  #Grensebetingelse for y=0
            p[i, Ny-1] = 0  #Grensebetingelse for siste y-punkt
        for j in range(Ny):
            p[0, j] = 0   #Grensebetingelse for x=0
            p[Nx-1, j]= 0   #Grensebetingelse for siste x-punkt
        return p


def advance_vectorized(self, p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n,␣
↪qL_values, rho_values, step1=False):
    """
    Vektorisert funksjon for å beregne differenslikning for pi,j,n+1
    Args:
        p (array) - trykk i punkt i tid n+1
        p_n (array) - trykk i punkt i tid n
        p_nm1 (array) - trykk i punkt i tid n-1
        xv (array) - x-koordinater
        yv (array) - y-koordinater
        Cx2 (float) - dt^2/dx^2
        Cy2 (float) - dt^2/dy^2
        dt (float) - tid mellom hvert punkt
        t (array) - tid
        n (int) - tidstep
        qL_values (array) - bølgefart i punkt
        rho_values (array) - massetetthet i punkt
    """
    b = self.b
    f = self.f
    rhoL_a = rho_values[1:-1, 1:-1] #Hjelpevariabel
    qL_a = qL_values[1:-1, 1:-1] #Hjelpevariabel
    qL_ip1 = qL_values[2:, 1:-1] #Hjelpevariabel
    qL_im1 = qL_values[:-2, 1:-1] #---
    qL_jp1 = qL_values[1:-1, 2:] #----
    qL_jm1 = qL_values[1:-1, :-2]
    p_ip1 = (qL_ip1+ qL_a) * (p_n[2:, 1:-1] - p_n[1:-1, 1:-1])
```

```
        pq_im1 = (qL_im1+qL_a) * (p_n[1:-1, 1:-1]- p_n[:-2, 1:-1])
        pq_jp1= (qL_jp1+qL_a) * (p_n[1:-1, 2:] - p_n[1:-1, 1:-1])
        pq_jm1= (qL_jm1+qL_a) * (p_n[1:-1, 1:-1] - p_n[1:-1, :-2])
        f_values = f(xv, yv, t[n], dt)
        f_values = f_values[1:-1, 1:-1]
        if step1: #Spesiell likning for første tidsteg
            p[1:-1, 1:-1] = (rhoL_a)**(-1)*0.5*(rhoL_a*2*p_n[1:-1, 1:-1] +␣
↪Cx2*0.5*(p_ip1 - pq_im1) + Cy2*0.5*(pq_jp1 - pq_jm1)) + dt**2*f_values
        else:
            p[1:-1, 1:-1] = (rhoL_a+0.5*b*dt)**(-1)*((0.5*b*dt-rhoL_a)*p_nm1[1:
↪-1, 1:-1] + 2*rhoL_a*p_n[1:-1, 1:-1] + Cx2*0.5*(p_ip1 - pq_im1) + Cy2*0.
↪5*(pq_jp1 - pq_jm1))+dt**2*f_values
        return p

    def adv_solver2dvectorized(self, x0=0, y0=0):
        """
        Funksjon for å løse bølgelikning i 2D med vektorisert kode
        Args:
            x0 (float) - Startverdi for x-koordinat
            y0 (float) - Startverdi for y-koordinat
        Returnerer:
            p_data (list) - Liste med trykkverdier for hver tid
            x (array) - x-koordinater
            y (array) - y-koordinater
            t (array) - tid
        """
        #Hjelpevariabler fra konstruktor
        K = self.K
        I = self.I; f = self.f; qL=self.qL
        dx = self.dx; dy = self.dy
        Nx = self.Nx; Ny = self.Ny; Nt = self.Nt
        xn = self.xn; yn = self.yn; tn = self.tn
        rho2 = self.rho2
        advance_vectorized = self.advance_vectorized;␣
↪boundary_conditions_vectorized=self.boundary_conditions_vectorized

        #Arrays
        x = np.linspace(x0, xn, Nx+1); dx = x[1]-x[0]
        y = np.linspace(y0, yn, Ny+1); dy = y[1]-y[0]

        #Arrays for broadcasting
        xv = x.reshape((x.size, 1)); yv = y.reshape((1, y.size))

        p = np.zeros((Nx+1, Ny+1)); p_n = np.zeros((Nx+1, Ny+1)); p_nm1 = np.
↪zeros((Nx+1, Ny+1))

        #Loop for rho2
```

```
    rho_values = np.zeros((xv.shape[0], yv.shape[1]), dtype=np.float64)
    for i in range(len(x)):
        for j in range(len(y)):
                rho_values[i, j] = rho2(x[i], y[j]) # was changed from self.
↪p2
    #Loop for qL
    qL_values = np.zeros((xv.shape[0], yv.shape[1]), dtype=np.float64)
    for i in range(len(x)):
        for j in range(len(y)):
                qL_values[i, j] =qL(x[i], y[j])  # Apply qL element-wise to␣
↪xv and yv

    #Sjekker for gyldige verdier
    if np.max(qL_values)<= 0 or np.isnan(np.max(qL_values)):
        raise ValueError("Feil i qL_values | <=0 elr NaN")
    dx_dy = np.array([dx, dy], dtype=np.float64)  # Convert to NpmPy array
    if np.min(dx_dy)<= 0 or np.isnan(np.min(dx_dy)):
        raise ValueError("Feil i dx, dy | <=0 elr NaN")

    B = 0.4 #Sikkerhetsfaktor
    dt = B *np.min(dx_dy)/np.sqrt((np.max(qL_values)) ␣
↪#Stabilitetsbetingelse
    print(dt)
    if dt<= 0 or np.isnan(dt):
        raise ValueError("Feil i dt | <=0 elr NaN")

    Nt = int(round(tn / dt))
    t = np.linspace(0, tn, Nt+1); dt = t[1] - t[0]  #I tillfelle␣
↪avrundingfeil

    #Courant tall
    Cx2 = (dt/dx)**2
    Cy2 = (dt/dy)**2


    p_data = []  #Trykk over mesh for hver tid

    # initial betingelse
    p[:, :] = I(xv, yv)
    p_data.append(p.copy())
    p_nm1, p_n, p = p_n, p, p_nm1

    #Første tidsssteg
    n = 0
    p = advance_vectorized(p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n,␣
↪qL_values, rho_values, step1=True)
    p = boundary_conditions_vectorized(p)
```

```
        p_data.append(p.copy())
        p_nm1, p_n, p = p_n, p, p_nm1

        #Loop over tid
        for n in range(1, Nt + 1):  # Corrected range syntax
            p = advance_vectorized(p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n,␣
↪qL_values, rho_values, step1=False)
            p = boundary_conditions_vectorized(p)
            p_data.append(p.copy())
            p_nm1, p_n, p = p_n, p, p_nm1

        return p_data, x, y, t


def animate(p_data, x, y, t, save=False, file=None, skipframes=1, interval=10):
    """
    Plotterer trykk over mesh for hver tid
    Args:
        p_data (list) - Liste med trykkverdier for hver tid
        x (array) - x-koordinater
        y (array) - y-koordinater
        t (array) - tid
        save (bool) - Om animasjonen skal lagres som mp4
        file (str) - Filnavn for lagring av animasjon
        skipframes (int) - Antall frames som skal hoppes over i animasjonen
    """

    fig = plt.figure(figsize=(8, 8))
    ax = plt.axes(projection='3d')
    plim = (np.min(p_data), np.max(p_data))
    X, Y = np.meshgrid(x, y)
    def update(frame): #Oppdateringsloop for animasjonen
        ax.clear()
        ax.contour3D(X, Y, p_data[frame], 50, cmap="cool") #Plotter␣
↪trykkverdier
        ax.set_title(f"Time: {t[frame-1]:.3f}") #Tittel med tid
        ax.set_xlabel("X [m]")
        ax.set_ylabel("Y [m]")
        ax.set_zlabel("P [Pa]")
        ax.set_zlim(plim)

    if save==True: #Lagrer animasjonen som gif hvis sann
        save_count=len(p_data)
        writer="PillowWriter" #Writer for gif saving
    else:
        save_count, writer = None, None
```

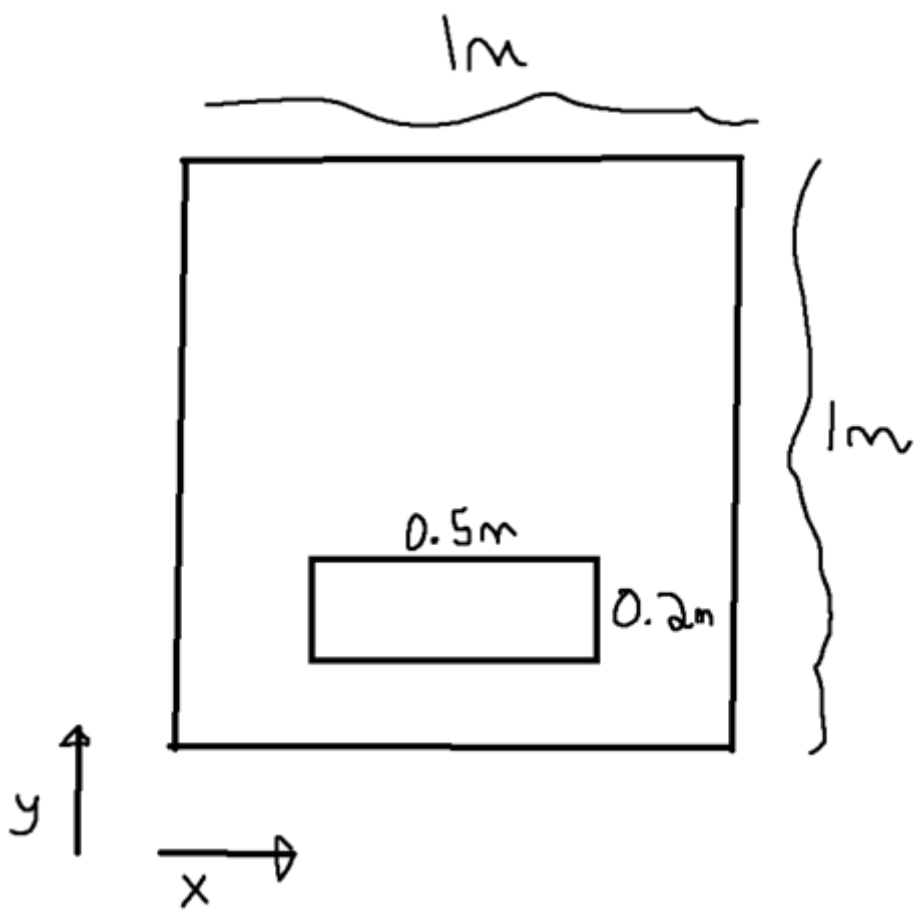
```
    anim = FuncAnimation(plt.gcf(), update, frames=range(0, len(p_data),␣
↪skipframes), interval=interval, save_count=save_count) #Bruker FuncAnimation␣
↪for å lage animasjonen

    if save == True:
        anim.save(f"{file}.gif", writer=writer) #Lagring
    plt.show()

def breaking(p_data, tensilelim, compresslim, t):
        """
        Deler opp i tre deler og sjekker om trykkverdier er over eller under␣
↪grenseverdier
        Args:
            p_data (list) - Liste med trykkverdier for hver tid.
            tensilelim (float) - Øvre grenseverdi.
            compresslim (float) - Nedre grenseverdi.
            t (array) - Tidspunkter.

        Returnerer:
            results (bool) - True hvis trykkverdier er innenfor grenseverdier,␣
↪False ellers.
            vals2 (list) - Liste med min og max trykkverdier i de tre delene.
        """
        p_data = np.array(p_data)
        dt = t[1]-t[0]
        Nx, Ny = p_data[0].shape
        width = Nx // 3


        part1 = p_data[:, :, :width] #Vesntre del
        part2 = p_data[:,  :, width:2 * width]  #Midtre del
        part3 = p_data[:, :, 2 * width:]  #Høyre del

        tvals = np.zeros(3)
        parts = [part1, part2, part3]

        for i, part in enumerate(parts, start=0):  #Loops over delene
            last_satisfied_time = -10 #Initialiserer med -10
            for n in range(p_data.shape[0], ):  #Loop over tid
                if np.any(np.logical_or(part[n] > tensilelim, part[n] <␣
↪compresslim)): #Sjekker om trykkverdier er over eller under grenseverdier
                    last_satisfied_time = n*dt #Regner ut tid
                    break
            tvals[i] = last_satisfied_time
        print(tvals)
        if np.any(tvals<0): #Hvis noen av delene er mindre enn null (altså␣
↪initialisert verdi -10) -> sprekk ikke formet
```

```
            results=False
        else:
            results=True #Returnerer sann hvis sprekk er formet

        least = [np.min(part) for part in parts] #Finner min og max i hver del
        biggest = [np.max(part) for part in parts]
        minval = np.max(least) #Minste av de største
        maxval = np.min(biggest) #Største av de minste
        vals2 = [minval, maxval]

        return results, vals2
```

3 Analyse og modellering:

[7]:
```
Image(filename='C://users//toosi//Pictures/Grunnmodell.png')
```

[7]:



4 Bevegelseslikning og kraftuttrykk

Modellen tar utgangspunkt i en 1mx1mx1m betong vegg. Først skal vi begynne med å modellere et 2 dimensjonalt system, så tar vi steget opp til 3D. Kraften The Flash utøver på veggen virker på et begrenset område som avhenger av dimensjonene til systemet vårt. I 2D systemet lar vi kraften virke på $x \in [0.25, 0.75], y \in [0.1, 0.3]$ i 3D lar vi kraften virke på $x \in [0.25, 0.75], y \in [0.1, 0.3], z \in [0.4, 0.6]$.Vi krever også at The Flash må sprekke betongen i løpet av t=2s. Kraften modellerer vi som en tidsavhengig oscillerende kraft:

$$F_{flash} = F\cos(\omega_f t)$$

Der ω_f er resonansfrekvensen til betong. Grunnfrekvensen f i et system med en bølgehastighet c (3000m/s for betong)[4] og lengde L er gitt ved:

$$f = \frac{c}{2L} = 1500Hz$$

For å gå videre med teorien må vi introdusere noen likninger: Utgangspunktet er bevegelseslikningen for et elastisk materiale:

$$\varrho\frac{\partial^2 u}{\partial t^2} = \nabla \cdot \sigma + \varrho\mathbf{f}$$

Der u er vektorfeltet til forskyvningene, ϱ er massetettheten til mediet, σ er stress tensoren og f er kroppskrefter (altså ikke-kontakt krefter som gravitasjon, elektrisk, osv.). Og Hooke's generaliserte lov:

$$\sigma = K\nabla \cdot \mathbf{uI} + G\left(\nabla\mathbf{u} + (\nabla\mathbf{u})^T - \frac{2}{3}\nabla \cdot \mathbf{uI}\right)$$

Der K er mediets bulkmodul (K=17Gpa i betong), I er identitets tensoren, og G er skjærmodul. Sammen gir disse to lovene informasjon om hvordan forskyvningene av partiklene i et media avhenger av mediets egenskaper, tid og rom. Setter vi disse to likningene sammen, får vi Navier likningen:

$$\varrho\frac{\partial^2\mathbf{u}}{\partial t^2} = \varrho\mathbf{g} + \left(K + \frac{1}{3}G\right)\nabla(\nabla \cdot \mathbf{u}) + G\nabla^2\mathbf{u}$$

Her gjør vi en akustisk tilnærming for modellen vår, og antar at skjærmodulen, G=0. Det vil si at vi tilnærmer betong som en væske! Grunnen til at dette kalles for en akustisk tilnærming er fordi når G=0 kan det ikke oppstå transversale bølger, som betyr at vi modellerer longitudinale bølger , som er slik lydbølger propagerer seg. Navier/bevegelseslikningen blir da:

$$\varrho\frac{\partial^2\mathbf{u}}{\partial t^2} = \nabla(K\nabla \cdot \mathbf{u})$$

For å relatere dette tilbake til veggen og når den sprekker introduserer vi trykk, p, som blir (under tilnærmingen):

$$p = -K\nabla \cdot \mathbf{u}$$

Setter vi p inn i bevegelseslikningen får vi, og gjør en antagelse om at $-\varrho^{-2}\nabla\varrho \approx 0$ (altså at σ varierer relativt tregt i rommet), så får vi:

$$\frac{\partial^2 p}{\partial t^2} = \frac{K}{\varrho}\nabla^2 p + s$$

Der s er en driver. Denne likningen skal vi forsøke å modellere numerisk. For å gjøre det bruker vi finit-forskjeller-metoden (Finite difference method). Det gir oss:

$$p_{i,j}^{n+1} = -p_{i,j}^{n-1} + 2p_{i,j}^{n} + \frac{K}{\varrho}^2 \Delta t^2 [D_x D_x p + D_y D_y p]_{i,j}^{n} + dt^2 s$$

(For n=0 setter vi $p_{i,j}^{n-1} = p_{i,j}^{n}$ siden $D_t p_{i,j}^{n}$=0 som initialbetingelse). Til å begynne med i analysen setter $\frac{K}{\varrho} = c^2$, og setter c=3000m/s, som er longitudinal bølgefart i betong.

[8]:
```
def advance_vectorized(f, p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n,␣
↪rho_values, step1=False, c=3000):
        """
        Vektorisert funksjon for å beregne differenslikning for pi,j,n+1
        Args:
            p (array) - trykk i punkt i tid n+1
            p_n (array) - trykk i punkt i tid n
            p_nm1 (array) - trykk i punkt i tid n-1
            xv (array) - x-koordinater
            yv (array) - y-koordinater
            Cx2 (float) - dt^2/dx^2
            Cy2 (float) - dt^2/dy^2
            dt (float) - tid mellom hvert punkt
            t (array) - tid
            n (int) - tidstep
            qL_values (array) - bølgefart i punkt
            rho_values (array) - massetetthet i punkt
        """
        #Hjelpevariabler
        rhoL_a = rho_values
        pq_ip1 = (p_n[2:, 1:-1] - p_n[1:-1, 1:-1])
        pq_im1 = (p_n[1:-1, 1:-1]- p_n[:-2, 1:-1])
        pq_jp1= (p_n[1:-1, 2:] - p_n[1:-1, 1:-1])
        pq_jm1= (p_n[1:-1, 1:-1] - p_n[1:-1, :-2])

        f_values = f(xv, yv, t[n], dt) #Kraft for alle punkter
        f_values = f_values[1:-1, 1:-1] #Kraft for indre punkter

        #Spesiell likning for første tidsteg
        if step1:
            p[1:-1, 1:-1] = (rhoL_a)**(-1)*0.5*(rhoL_a*2*p_n[1:-1, 1:-1] +␣
↪(Cx2*0.5*(pq_ip1 - pq_im1) + Cy2*0.5*(pq_jp1 - pq_jm1))) + dt**2*f_values
        else:
            p[1:-1, 1:-1] = (rhoL_a)**(-1)*((-rhoL_a)*p_nm1[1:-1, 1:-1] +␣
↪rhoL_a*p_n[1:-1, 1:-1] + c**2*(Cx2*0.5*(pq_ip1 - pq_im1) + Cy2*0.5*(pq_jp1 -␣
↪pq_jm1)))+dt**2*f_values
        return p
```

5 Stabilitet

I koden over har vi gjort en forenkling ved å introdusere Courant-tallet: $C_x = (\frac{dt}{dx}c)^2$ og tilsvarende for y. Dette tallet er viktig fordi $C \leq 1$, atlså $dt \leq \frac{dx}{c}$ er et stabilitetskriterie[6]. I tillegg har vi lagt til en tryggshetsfaktor (som er egentlig bare nødvendig når c varierer). Det er her også verdt å nevne at med en frekvens lik f=1500Hz, trenger vi at tidssteget vårt er $dt_{min} = \frac{1}{2f} = \frac{1}{3000Hz} = 3.33*10^{-4}$, fra Nyquist teorem. Er dette kravet ikke oppfylt oppstår aliasing, som fører til at kraften modelleres feil.

6 Driver og grensebetingelser

Siste steg er å finne et uttrykk for driveren s. s er i enheter Pa/t^2, altså det er en akselerasjon for trykket p. Trykket fra driveren kaller vi p_s.

$$p_s = \frac{F_{flash}}{A}$$

$$s(t) = \frac{\partial^2 p_s}{\partial t^2} = \frac{1}{A}\frac{\partial^2 F_{flash}}{\partial t^2}$$

$$s(t) = -\frac{1}{A}\omega_f^2 F \cos(\omega t)$$

[ ]:
```
def f(F, x, y, t, dt, wF=1500*2*np.pi):
    """
    Funksjon for kraft i punkt
    Args:
        x, y - Koordinater til punkt
        t - tid
        wF - vinkelfrekvens
    Returnerer:
        Kraft i punkt

    """
    A = (0.75-0.25)*(0.3-0.1) #Areal for kraft
    f =-1/(A)*F*wF**2*np.cos(wF*t) #Kraft i punkt
    return np.where((0.25 <= x) & (x <= 0.75) & (0.1 <= y) & (y <= 0.3), f, 0)␣
↪#Område hvor kraft virker
```

Vi er også nødt til å definere noen grensebetingelser for betong veggen/blokken vår. På grensen av blokken er det atmosfærisk trykk, slik at grensebetingelse er p=p0=101.3Pa. Jeg fikk litt problemer når jeg satt den betingelsen, så vi velger heller p=0. Det har lite å si for analysen, siden dette er konstant og den er svært liten.

[ ]:
```
def boundary_conditions_vectorized(p, Nx, Ny):
    """
    Grensebetingelser
    Args:
        p (array) - trykk i punkt
        Nx (int) - Antall x punkter
```

```
        Ny (int) - Antall y punkter
    Returnerer:
        p (array) - trykk i punkt med verdi 0 i ytre punkter
    """



    Nx, Ny =p.shape
    for i in range(Nx):
        p[i, 0] = 0  #Grensebetingelse for y=0
        p[i, Ny-1] = 0  #Grensebetingelse for siste y-punkt
    for j in range(Ny):
        p[0, j] = 0  #Grensebetingelse for x=0
        p[Nx-1, j]= 0 #Grensebetingelse for siste x-punkt
    return p
```

Nedenfor er den generelle funksjonen for simulasjonen, som setter sammen funksjonene over:

7 Main loop og sprekkbetingelser

[11]:
```
def _adv_solver2dvectorized(I, f, dx, dy, Nx, Ny, Nt, xn, yn, tn, rho, x0=0,␣
 ↪y0=0, K=28e+6, c=3000):
        """"
        Funksjon for å løse bølgelikning i 2D med vektorisert kode
        Args:
            I (function) - Initial betingelse
            f (function) - Kraft i punkt
            dx (float) - Spacing
            dy (float) - Spacing
            Nx (int) - Antall x punkter
            Ny (int) - Antall y punkter
            Nt (int) - Antall tidspunkter
            xn (float) - x sluttpunkt
            yn (float) - y sluttpunkt
            tn (float) - tid sluttpunkt
        Returnerer:
            p_data (list) - Liste med trykkverdier for hver tid
            x (array) - x-koordinater
            y (array) - y-koordinater
            t (array) - tid
        """



        #Arrays
        x = np.linspace(x0, xn, Nx+1); dx = x[1]-x[0]
        y = np.linspace(y0, yn, Ny+1); dy = y[1]-y[0]
        xv = x.reshape((x.size, 1)); yv = y.reshape((1, y.size))
```

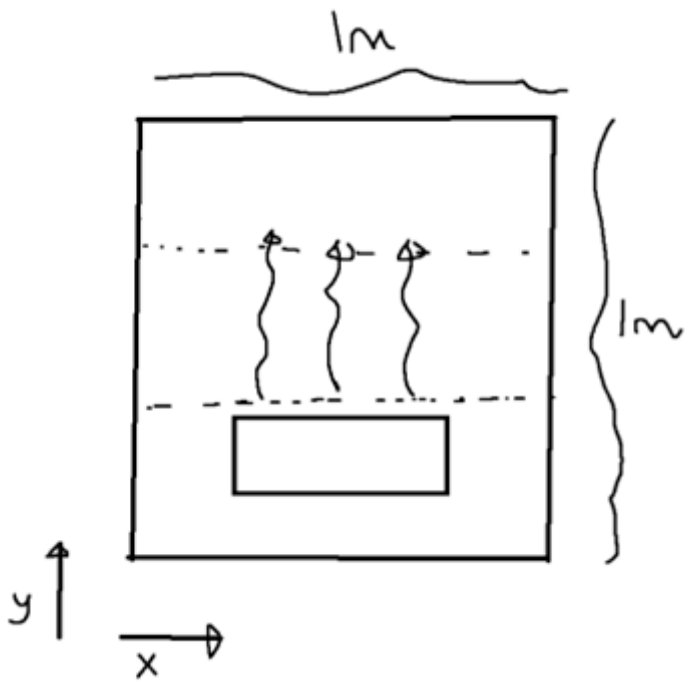
```
    p = np.zeros((Nx+1, Ny+1)); p_n = np.zeros((Nx+1, Ny+1)); p_nm1 = np.
↪zeros((Nx+1, Ny+1))

    #Loop for massetetthet
    rho_values = np.zeros((xv.shape[0], yv.shape[1]), dtype=np.float64)
    for i in range(len(x)):
        for j in range(len(y)):
                rho_values[i, j] = rho(x[i], y[j]) #

    dx_dy = np.array([dx, dy], dtype=np.float64)

    B = 0.4
    dt = B *np.min(dx_dy)/c #Stabilitetsbetingelse

    if dt<= 0 or np.isnan(dt):
        raise ValueError("Feil i dt | <=0 elr NaN")

    Nt = int(round(tn / dt))
    print(Nt); print(dt)
    t = np.linspace(0, tn, Nt+1); dt = t[1] - t[0]#Trygghet for␣
↪avrundingfeil

    Cx2 = (c*dt/dx)**2 #Courant tall
    Cy2 = (c*dt/dy)**2


    p_data = []  #Trykk over mesh for hver tid

    p[:, :] = I(xv, yv)  #Initial betingelse
    p_data.append(p.copy())
    p_nm1, p_n, p = p_n, p, p_nm1 #Bytter referanse

    #Første tidsssteg
    n = 0
    p = advance_vectorized(p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n, rho,␣
↪step1=True)
    p = boundary_conditions_vectorized(p)
    p_data.append(p.copy())
    p_nm1, p_n, p = p_n, p, p_nm1


    #Loop over tid
    for n in range(1, Nt + 1):
        p = advance_vectorized(p, p_n, p_nm1, xv, yv, Cx2, Cy2, dt, t, n,␣
↪rho, step1=False)
        p = boundary_conditions_vectorized(p)
        p_data.append(p.copy())
```

```
            p_nm1, p_n, p = p_n, p, p_nm1

        return p_data, x, y, t
```

Nå kan vi begynne å lete etter et tallsvar. Det maksimale trykket (compressive strength) som betong kan tåle er –17 til -28Mpa[9]. Merk at dette er negativt fordi trykk får et negativt fortegn når volumet minskes. Det minste trykket (tensile strength) som betong kan tåle er 2 til 5Mpa[9]. Det er viktig å notere at det finnes en mye større rekkevidde av tallverdier på nettet, men størrelsesorden er alltid i Mpa. Vi velger respektivt -30Mpa og 3Mpa som verdiene våre. Men vi kan ikke si oss fornøyde helt enda. Vi kan ikke bare finne et punkt som er over eller under disse verdiene for å si at betongen har sprukket. Vi velger derfor heller å dele området i tre, på tvers av der bølgen propagerer. Så krever vi at det har formet seg en sprekk (trykket er over eller under trykkgrensene) i hver av disse delene. Dette kan vi gjøre om til en funksjon, kalt breaking:

[12]:
```
Image(filename='C://users//toosi//Pictures/SprekkConcrete.png')
```

[12]:



[ ]:
```
def breaking(p_data, tensilelim, compresslim, t):
        """
        Deler opp i tre deler og sjekker om trykkverdier er over eller under␣
 ↪grenseverdier
        Args:
            p_data (list) - Liste med trykkverdier for hver tid.
            tensilelim (float) - Øvre grenseverdi.
            compresslim (float) - Nedre grenseverdi.
            t (array) - Tidspunkter.
```

```
    Returnerer:
        results (bool) - True hvis trykkverdier er innenfor grenseverdier,␣
↪False ellers.
        vals2 (list) - Liste med min og max trykkverdier i de tre delene.
    """

    p_data = np.array(p_data)
    dt = t[1]-t[0]
    Nx, Ny = p_data[0].shape  #
    width = Nx // 3  # Bredde for hver del

    # Definerer 3 deler
    part1 = p_data[:,:,:width]  # Venstre del
    part2 = p_data[:, :,  width:2 * width] #Midtre del
    part3 = p_data[:, :, 2 * width:]  # Høyre del

    #Lager array for å lagre siste tidssteg der grensebetingelse er oppfylt
    tvals = np.zeros(3)
    parts = [part1, part2, part3]

    #Looper over alle deler og sjekker om grensebetingelse er oppfylt
    for i, part in enumerate(parts, start=0):
        last_satisfied_time = -10  #Default verdi
        for n in range(p_data.shape[0], ):  # Loop over tid
            if np.any(np.logical_or(part[n, :, :] > tensilelim, part[n, :, :
↪] < compresslim)): #Betingelser
                last_satisfied_time = n*dt  #Oppdaterer siste tid der␣
↪grensebetingelse er oppfylt
                break
        #Lagrer siste tid med index for del
        tvals[i] = last_satisfied_time
    print(tvals)
    #Hvis noen betingelser ikke er oppfylt returner False
    if np.any(tvals<0): #Hvis noen av delene er mindre enn null (altså␣
↪initialisert verdi -10) -> sprekk ikke formet
        results=False
    else:
        results=True

    #Lager liste med min og max trykkverdier i hver del
    least = [np.min(part) for part in parts]
    biggest = [np.max(part) for part in parts]

    #Finner minste av de minste og største av de største
    minval = np.max(least)
    maxval = np.min(biggest)
```

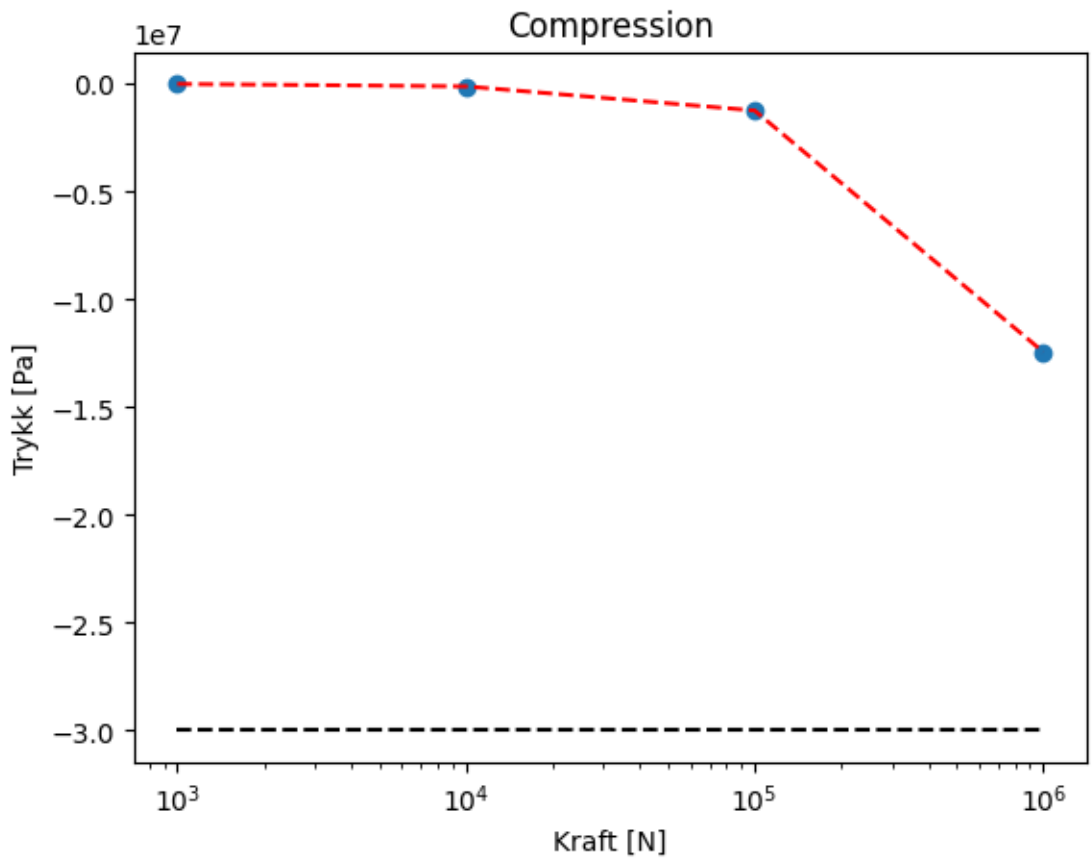
```
        vals2 = [minval, maxval]

        return results, vals2
```

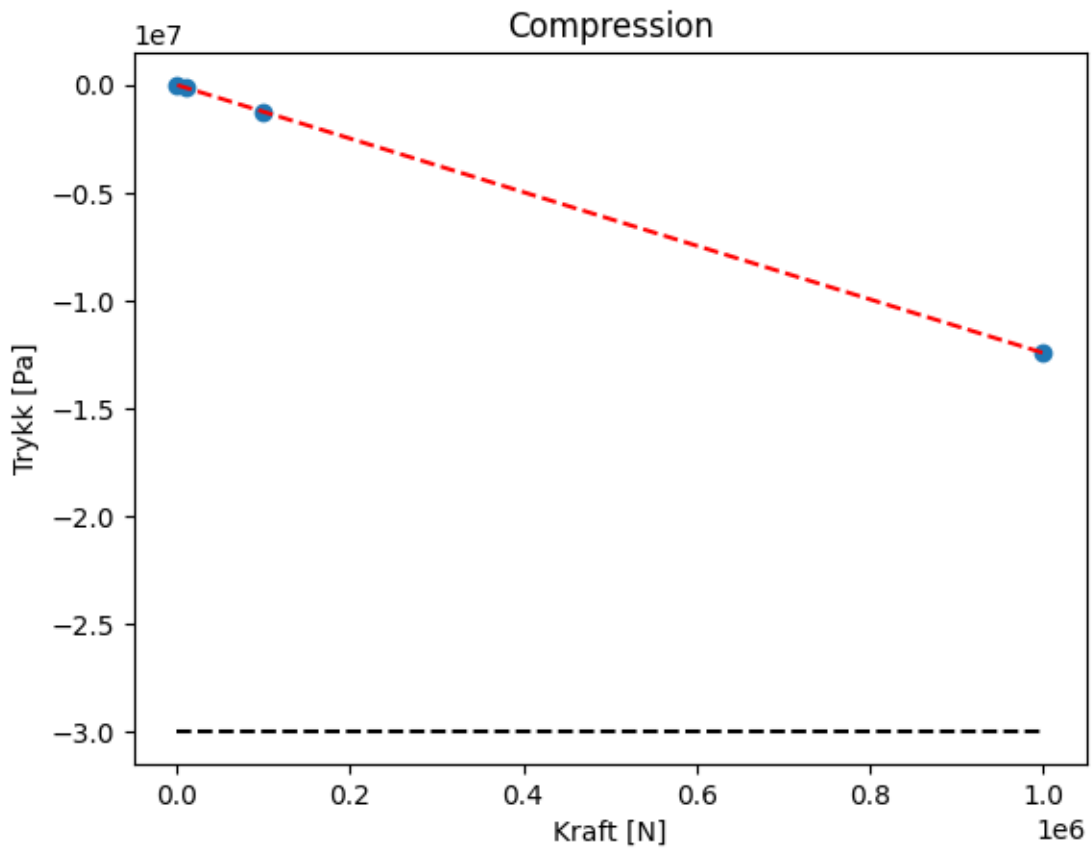
Også kan vi lage en funksjon som looper over breaking for forskjellige størrelserordener av F (modulus til kraftuttryket):

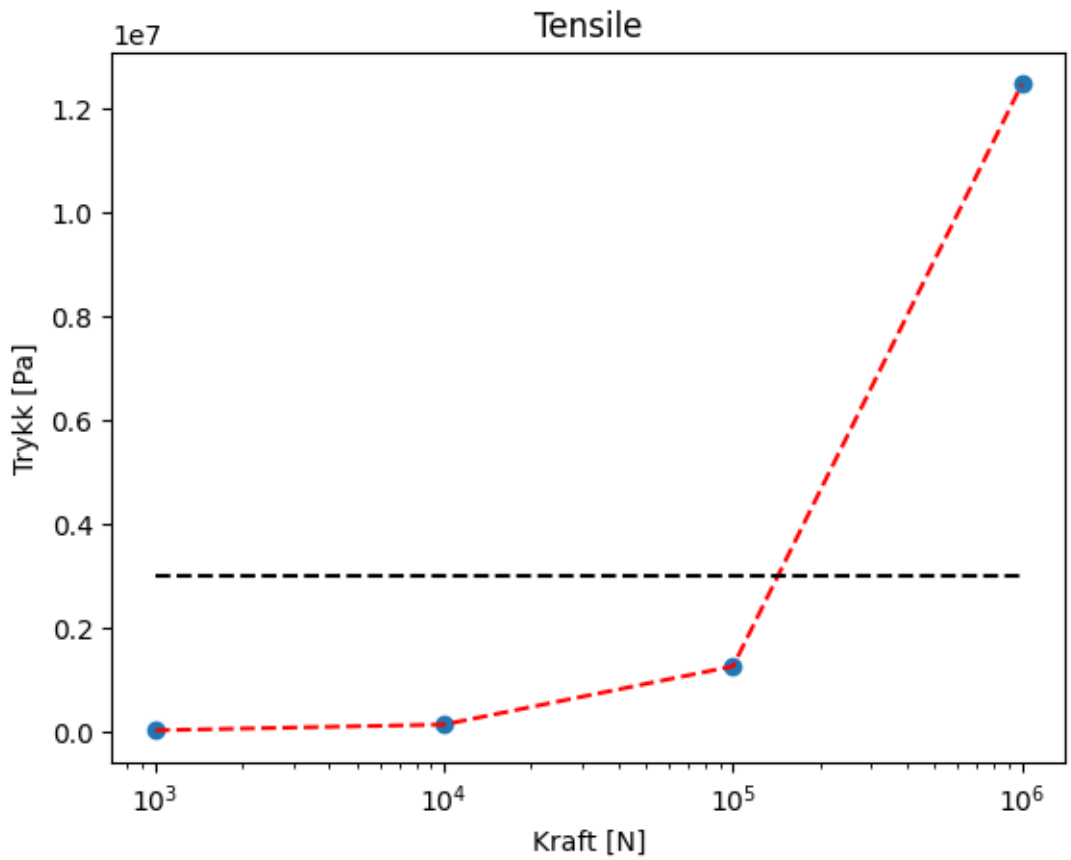
```
[ ]: b = wavesolver(b=0, F=100, dx=1, dy=1, Nx=10, Ny=10, Nt=5, xn=1, yn=1, tn=2)
p, x, y, t, = b.adv_solver2dvectorized()
animate(p, x, y, t, save=True, file="2Dnumba-F=100", skipframes=1000,␣
 ↪interval=100)

f = [10**i for i in range(3, 10)]
compresslim = -30e+6; tensilelim=3e+6
def foo2(b, f, compresslim, tensilelim, constant=False):
    """
    Funksjon for å kjøre wavesolver og sjekke om grensebetingelser er oppfylt
    Args:
        b (float) - Dempingskoeffisient
        f (list) - Liste med kraftmodulus
        compresslim (float) - Nedre grenseverdi
        tensilelim (float) - Øvre grenseverdi
        constant (bool) - Om massetetthet og bølgefart skal være konstant
    Returnerer:
        least (list) - Liste med trykkverdi nærmest compressbetingelse
        biggest (list) - Liste med trykkverdi nærmest tensilebetingelse
        fnew (list) - Liste med kraftmodulus
    """
    fnew = []
    least = []
    biggest = []
    #loop over kraftmodulus
    for i in range(len(f)):
        c = wavesolver(b, F=f[i], dx=1, dy=1, Nx=10, Ny=10, Nt=10, xn=1, yn=1,␣
 ↪tn=2) #Ny instans av wavesolver
        if constant:
            c.change_speed(3000, True) #Setter konstant fart og massetetthet␣
 ↪hvis ønskelig
        p, x, y, t = c.adv_solver2dvectorized() #Løser bølgelikning
        results, vals = breaking(p, tensilelim, compresslim, t) #Sjekker␣
 ↪grensebetingelser
        results = results #True hvis grensebetingelser er oppfylt
        least_element = vals[0] #Nærmest compressbetingelse
        biggest_element = vals[1]  #Nærmest tensilebetingelse

        #legger til verdier i lister
        biggest.append(biggest_element)
```

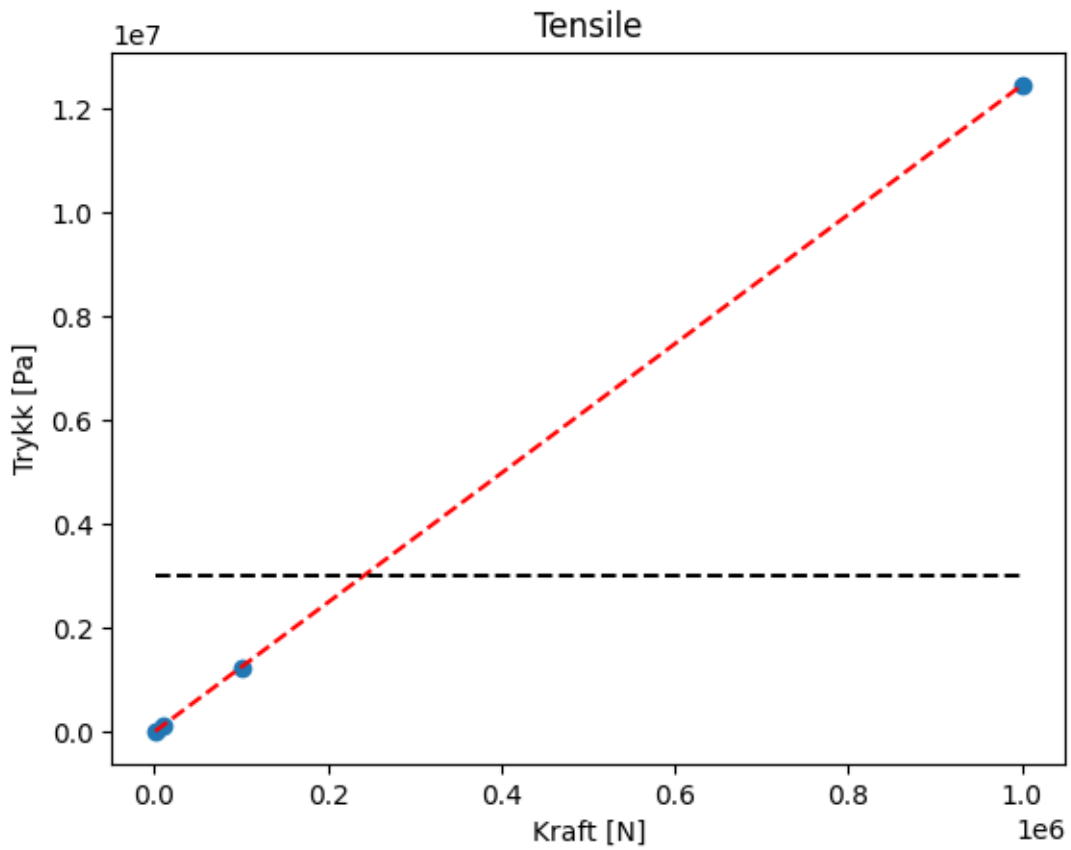
```
        least.append(least_element)
        fnew.append(f[i])
        if results == True:
            break
    return least, biggest, fnew


def plotting(vals, fnew, lim, title):
    """
    Funksjon for å plotte trykkverdier som funksjon av kraft
    """

    plt.plot(fnew, vals, "o")
    plt.plot(fnew, vals, "r--")
    x = np.linspace(np.min(fnew),np.max(fnew), 1000)
    plt.plot(x, 0*x+lim, "k--")
    plt.xscale("log")
    plt.xlabel("Kraft [N]"); plt.ylabel("Trykk [Pa]")
    plt.title(title)
    plt.show()
    plt.xscale("linear")
    plt.plot(fnew, vals, "o")
    plt.plot(fnew, vals, "r--")
    x = np.linspace(np.min(fnew), np.max(fnew), 1000)
    plt.plot(x, 0*x+lim, "k--")
    plt.xlabel("Kraft [N]"); plt.ylabel("Trykk [Pa]")
    plt.title(title)
    plt.show()
least, biggest, fnew = foo2(0, f, compresslim, tensilelim)
plotting(least, fnew, compresslim, "Compression")
plotting(biggest, fnew, tensilelim, "Tensile")
```

```
1.1212238211627765e-05
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[0.00133425 0.00493337 0.00945189]
```


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
10^3
10^4
10^5
10^6
Kraft [N]


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
0.0
0.2
0.4
0.6
0.8
1.0
1e6
Kraft [N]


Tensile
1e7
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Trykk [Pa]
10^3
10^4
10^5
10^6
Kraft [N]



[23]:
```
Image(filename='C://users//toosi//Desktop/2Dnumba-F=100.gif', width=600)
```

[23]: <IPython.core.display.Image object>

Figur 1: Animasjon av 2D simulering med F=100 og b=0. Området til venstre i plottet (se figur 2) er der hvor kraften tar effekt, og videre til høyre er hvordan bølgene sprer seg ut i 2D-planet.

I kraftplottene observerer vi først hvor mye mer resistant betong er til komprimering i forhold til tøyning. Trykket er langt forbi tøyningsgrensen før den begynner å nærme seg komprimeringsgrensen. Vi observerer også en lineær trend mellom kraft og trykkbetingelsene. Det vi har plottet her er ikke nødvendigvis den minste trykkverdien i betongen, det er trykket som er lengst unna grensebetingelsene i de tre delene (se minste av de største kommentar i koden).

Vi ser at The Flash må produsere mellom 10^5 og 10^6 Newton. Det er omtrent like mye kraft som 1-2 GE90-115B motorer[10]. Det er motorene som Boeing-777 fly bruker! Vi forsøker ikke å finne et mer nøyaktig tall enn dette, på grunn av den store rekkeviden på konstanter for betong. Det er verdt å bemerke seg at tiden betingelsene er oppfylt er forventet, der det tar lengst tid for sprekken til å forme seg i delen lengst unna der kraften utøves.

8 Dypere analyse: Varierende massetetthet

Vi velge å gå et steg mer nøyaktig i analysen vår ved å variere massetettheten $\sigma(x, y)$. Siden bølgehastigheten $c(x, y) = \sqrt{\frac{K}{p(x,y)}}$ er avhengig av massetettheten, så varierer også c.

Det endrer den partiell deriverte med hensyn på x til:

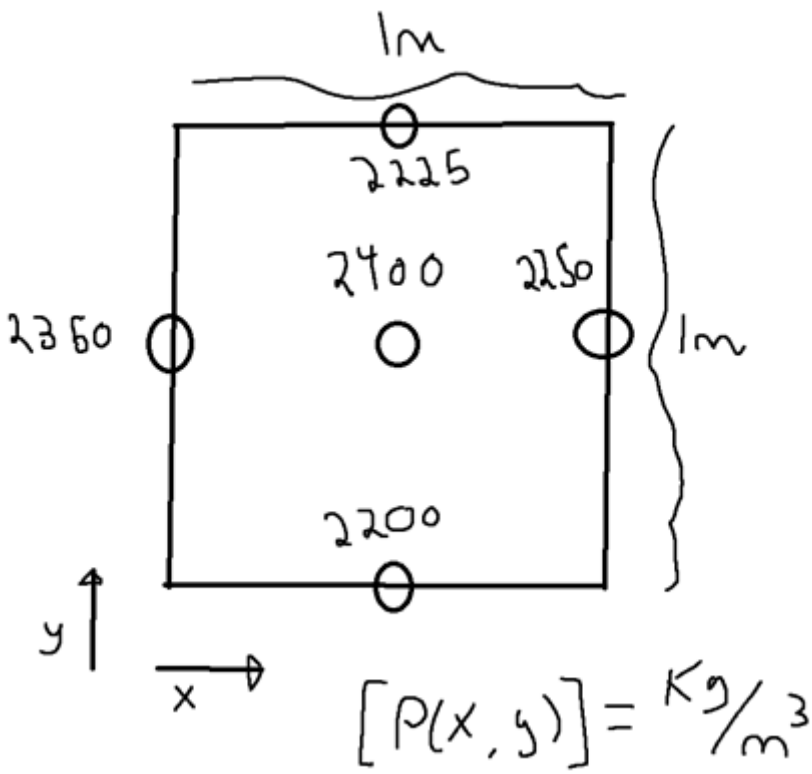
$$[\frac{\partial}{\partial x}(q(x)\frac{\partial p}{\partial x})]^n_{i,j} \approx \frac{1}{\Delta x^2}(\frac{1}{2}(q^n_{i+1,j} + q^n_{i,j})(p^n_{i+1} - p^n_i) - \frac{1}{2}(q^n_{i,j} + q^n_{i-1,j})(p^n_i - p^n_{i-1}))$$

Og tilsvarende for y og z. Der $q(x, y) = \frac{K}{p(x,y)} = c^2$. Merk at siden q(x, y) er definert slik, så har vi videre i modellen dratt c^2 ut fra Courant-tallet: $C_x = (\frac{dt}{dx}c)^2$, slik at det nå blir $C_x = (\frac{dt}{dx})$

For å bestemme massetettheten p(x, y) velger vi oss et par punkter med kjente verdier for massetetthet:

[ ]:
```
Image(filename='C://users//toosi//Pictures/Massetetthet.png')
```

[ ]:



For å finne p(x,y) basert på de kjente punktene våre, bruker vi invers avstands veiing (IDW):

$$\sigma(x, y, z) = \frac{\sum_{i=1}^n w_i \rho_i}{\sum_{i=1}^n w_i}$$

$$w_i = \frac{1}{d_i^p}$$

Der d_i er avstanden fra et vilkårlig punkt (x_i, y_i) til et kjent punkt (x, y). p er en parameter for hvor sterk vi vekter de nærmeste punktene. Vi setter p=2. Dette kan vi implmenetere som en funksjon (både for massetettheten og den varierende hastigheten). Merk at i funksjonen for massetetthet må vi sette en sjekk i tilfelle avstanden er null.

[ ]:
```
def rho2(points, dpoints, x, y, constant):
    """
    Funksjon for varierende massetetthet
    Args:
        points (array) - Array med punkter
        dpoints (array) - Array med massetetthet i punkter
        x (float) - x-koordinat
        y (float) - y-koordinat
        constant (bool) - Om massetetthet skal være konstant
    Returnerer:
        rho (float) - Massetetthet i punkt

    """
    for i in range(points.shape[0]): #Looper over punktene og sjekke om de er␣
 ↪like
        point = points[i]
        if (np.abs(point[0] - x) < 1e-5 and
            np.abs(point[1] - y) < 1e-5):
            return dpoints[i]
    d = np.zeros(points.shape[0], dtype=np.float64)  #Array for tetthet
    for k in range(points.shape[0]):
        dx = x - points[k, 0]  #x avstand
        dy = y - points[k, 1]  #y avstand
        d[k] = (dx**2 + dy**2 )**0.5  #Avstand
        if d[k] < 1e-5:#Toleranse for avstand
                return dpoints[k]
    w = np.zeros(d.shape[0])
    for i in range(d.shape[0]): #Regner ut vekter
            w[i] = 1 / (d[i]**2)
    w_sum = np.sum(w) #Summen av vektene
    rho = np.sum(dpoints * w) / w_sum #Regner ut massetetthet
    if constant ==True:
        return 2400
    return rho
```

[ ]:
```
def qL(self, x, y):
    """
    Funksjon for bølgefart:
    Args:
        x, y - Koordinater for punkt
    returnerer:
        Bølgefart i punkt
```

```
    """

    rho = self.rho2; K = self.K
    qL = K/rho(x, y) #Formel for bølgefart
    if self.constant==True: #Returnerer konstant bølgefart
        return self.c**2
    return qL
```

9 Dypere analyse: Demping

Vi har også glemt å inkludere noe svært viktig: demping. Dempingskoeffisienten i et harmonisk dempet system er gitt ved:

$$b = 2\zeta m\omega_0 = 4\zeta m\pi f$$

$$4 * 0.01 * 2400 * 1500kg/s = 904778kg/s \approx 1 * 10^6 kg/s$$

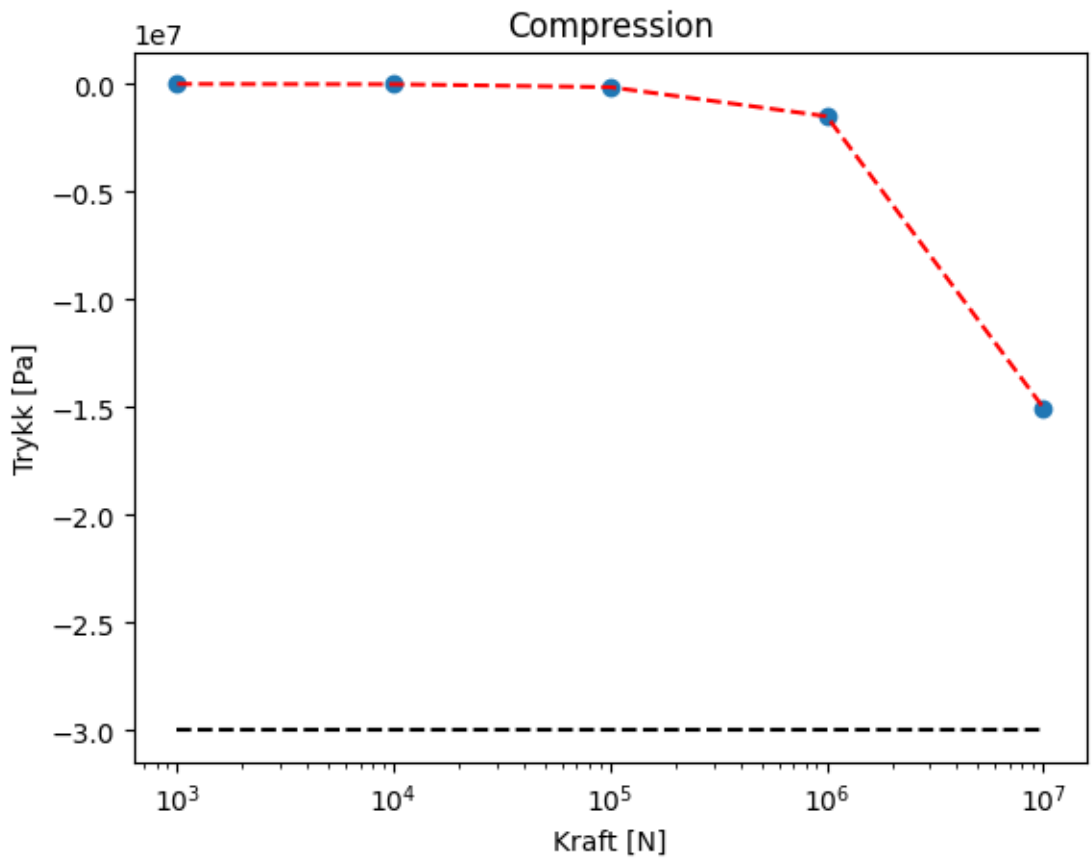
Der verdien $\zeta = 0.01$[7] har blitt brukt for dempingsforholdet, m=2400kg har blitt brukt ($\sigma = 2400kg/m^3$)[8]. Dempingskoeffisienten viser at det var viktig å beregne resonansfrekvensen i starten. Resonansfrekvensen vil bare ha høyere amplituderespons hvis tiden gått er lenger enn mange $\frac{1}{\gamma}$. Med så stor dempingskoeffisient får vi at $\frac{1}{\gamma} = \frac{2m}{b} = 0.005s$. Varigheten på simulasjonene t=2s er mye lenger! Resonans har derfor en betydelig i effekt i disse tilfellene.

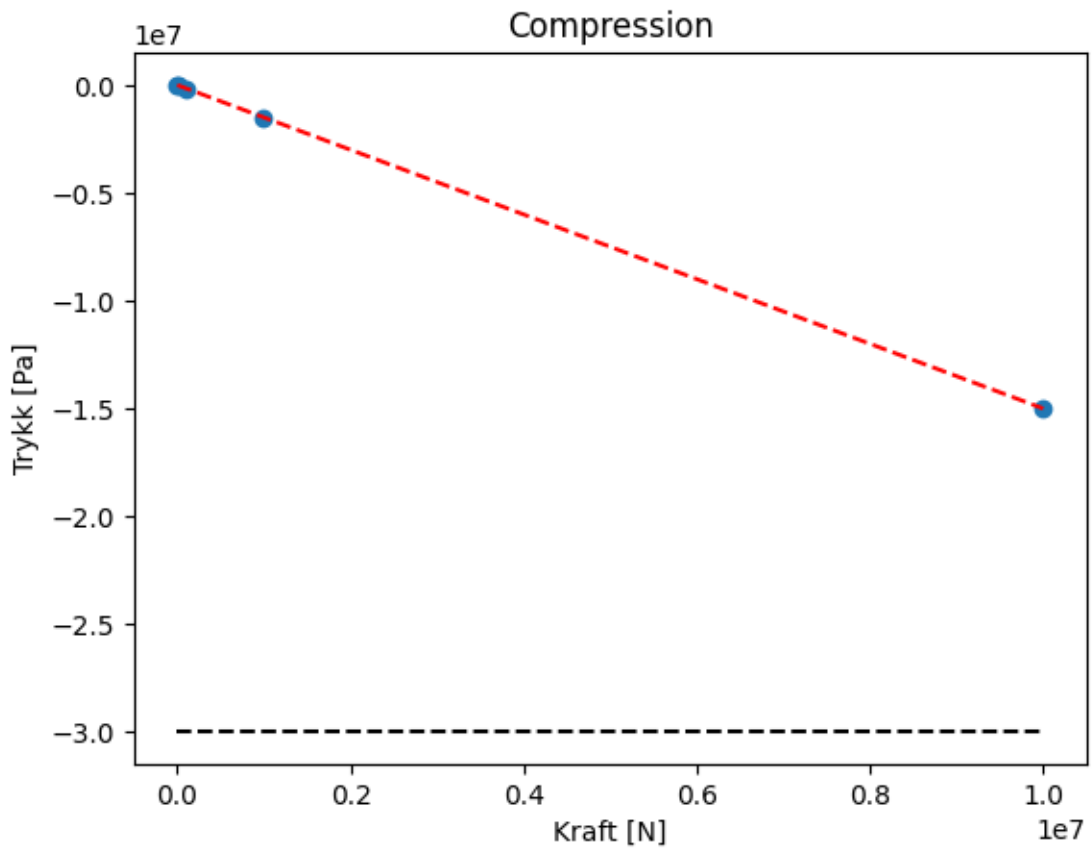
Dempingen endrer også litt på differenslikningene (merk dette er for konstant c for ryddighet, men hensikten er bare å vise hvordan b inngår i likningene).

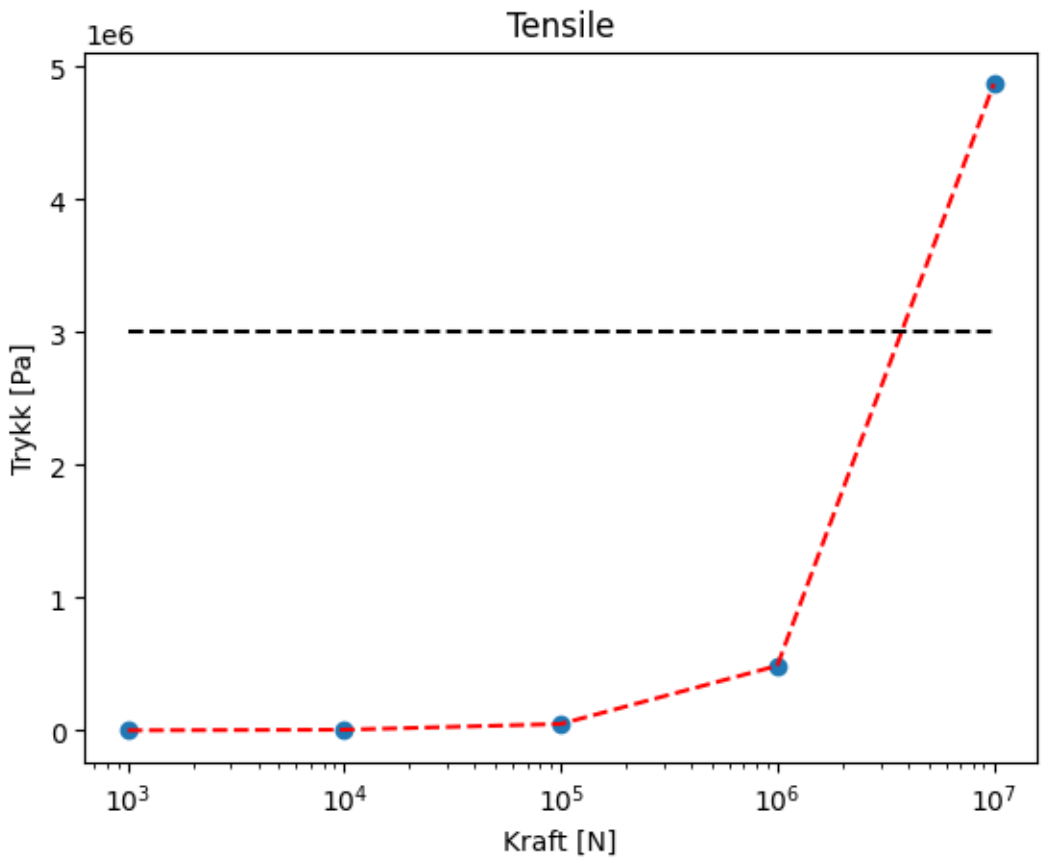
$$p_{i,j}^{n+1} = (\sigma + \frac{1}{2}b\Delta t)^{-1}((\frac{1}{2}b\Delta t - \sigma)p_{i,j}^{n-1} + 2\sigma p_{i,j}^n + C_x(p_{i+1,j}^n - 2p_{i,j}^n + p_{i-1,j}^n) + C_y(p_{i,j+1}^n - 2p_{i,j}^n + p_{i,j-1}^n)) + + \Delta t^2 s(t)$$

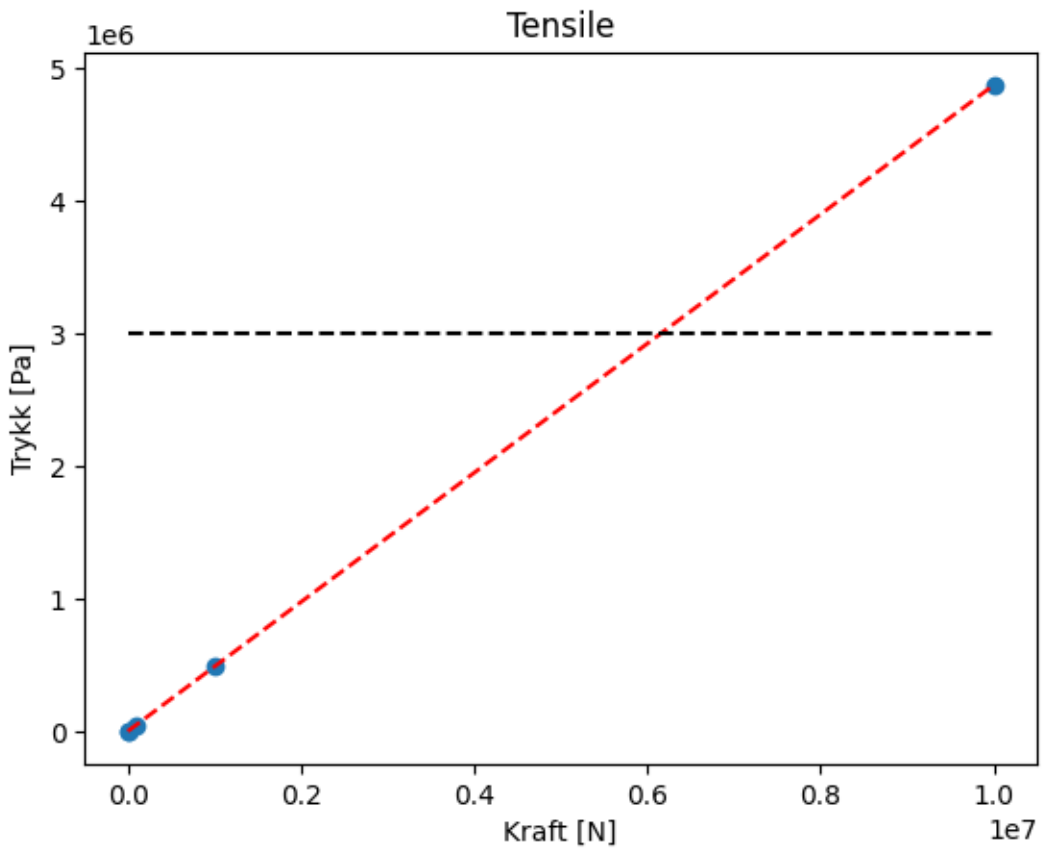
[ ]:
```
b = wavesolver(b=1e+6, F=1e+7, dx=1, dy=1, Nx=10, Ny=10, Nt=5, xn=1, yn=1, tn=2)
p, x, y, t, = b.adv_solver2dvectorized()
animate(p, x, y, t, save=True, file="2Ddempetnumba-F=1e+7", skipframes=1000,␣
 ↪interval=100)

f = [10**i for i in range(3, 16)]
least, biggest, fnew = foo2(1e+6, f, compresslim, tensilelim, constant=False)
plotting(least, fnew, compresslim, "Compression")
plotting(biggest, fnew, tensilelim, "Tensile")
```

```
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[-10. -10. -10.]
1.1212238211627765e-05
[ 1.25576728e-03 -1.00000000e+01 -1.00000000e+01]
1.1212238211627765e-05
[8.96976628e-05 1.26697949e-03 9.95644057e-03]
```


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
10^3
10^4
10^5
10^6
10^7
Kraft [N]


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
0.0
0.2
0.4
0.6
0.8
1.0
1e7
Kraft [N]


Tensile
1e6
5
4
3
2
1
0
Trykk [Pa]
10^3
10^4
10^5
10^6
10^7
Kraft [N]



[25]:
```
Image(filename='C://users//toosi//Desktop/2Ddempetnumba-F=1e+7.gif', width=600)
```

[25]:
```
<IPython.core.display.Image object>
```

Figur 2: Animasjon av dempet 2D simulering med F=1e+7 og b=1e+6. Utslagene til venstre er kraften The Flash utøver, linjene/fargene til høyre er utslagene som brer seg ut i rommet. De er mindre tydelig her sammenlignet med figur 1, på grunn av den store dempingskoeffisienten.

I kraftplottene observerer vi en lik lineær trend som uten dempingskoeffisienten, men dempingskoeffisienten har økt kraften med 1 størrelsesorden. Vi ligger nå altså i overkant av 10^6 Newton. Tidene sprekkene er formet er også forventet, der det med demping også tar lengre tid for sprekken å forme desto lenger unna den er kraften. Forskjellen er at her har demping skapt mye større forskjeller i tidene. Det er en forskjell på to størrelsesordener mellom del 1, og 2, 3: [8.96976628e-05 1.26697949e-03 9.95644057e-03]. Dette er også visualisert i figur 2, der vi observerer mye mindre utslag i punkter lenger unna kraften, sammenlignet med figur 1.

10 3D Modellering:

Nå kan vi ta steget opp til det 3 dimensjonale systemet. Det endrer på nesten ingenting av teorien, siden differenslikningene skaleres veldig greit opp til 3D. Det er lagt under i en klasse. Helst burde

3D og 2D blitt gjort om til en klasse, men vi velger å fortsette med en seperat klasse:

[ ]:
```
spec = [
    ('dx', float64),
    ('dy', float64),
    ('dz', float64),
    ('Nx', int32),
    ('Ny', int32),
    ('Nz', int32),
    ('Nt', int32),
    ('xn', float64),
    ('yn', float64),
    ('zn', float64),
    ('tn', float64),
    ('F', float64),
    ('K', float64),
    ('b', float64),
    ('constant', boolean),
    ('points', float64[:, :]),
    ('dpoints', float64[:]),
    ('c', float64)  # Add 'c' to the spec
]

@jitclass(spec)
class wavesolver3D:

    def __init__(self, b, F, dx, dy, dz, Nx, Ny, Nt, Nz, xn, yn, zn, tn,␣
 ↪K=28e+9):
        """
        Konstruktør for bølgeløser i 3D
        Args:
            b (float) - Dampingfaktor
            F (float) - Kraft
            dx, dy, dz (float) - Meshsteg i x, y og z retning
            Nx, Ny, Nz (int) - Antall meshsteg i x, y og z retning
            Nt (int) - Antall tidsteg
            xn, yn, zn (float) - Størrelse på mesh i x, y og z retning
            tn (float) - Tid
            K (float) - Bølgefart
        """
        self.dx = dx; self.dy = dy; self.dz = dz
        self.Nx = Nx; self.Ny = Ny; self.Nz = Nz; self.Nt = Nt
        self.xn = xn; self.yn = yn; self.zn=zn; self.tn = tn
        self.F = F; self.K = K; self.constant = False
        self.b=b
        self.c = 0.0
```

```
        self.points = np.array([[0.0, 0.5, 0.5], [1.0, 0.5, 0.5], [0.5, 0.5, 0.
↪5], [0.5, 0.0, 0.5], [0.5, 1.0, 0.5], [0.5, 0.5, 0.0], [0.5, 0.5, 1.0]],␣
↪dtype=np.float64)
        self.dpoints = np.array([2350.0, 2250.0, 2500.0, 2200.0, 2225.0, 2450.
↪0, 2300.0], dtype=np.float64)

    def set_mesh(self, dx, dy, dz, Nx, Ny, Nz, Nt):
        """
        Setter mesh
        Args:
            dx, dy, dz (float) - Steg i x, y og z retning
            Nx, Ny, Nz (int) - Antall Steg i x, y og z retning
            Nt (int) - Antall tidsteg

        """
        self.dx = dx
        self.dy = dy
        self.dz = dz
        self.Nx = Nx
        self.Ny = Ny
        self.Nz = Nz
        self.Nt = Nt

    def change_mesh(self, refine=False, coarse=False, factor=0.5):
        """
        Endrer meshsteg med en faktor"""
        if refine:
            self.dx=self.dx*factor #Refinerer mesh
            self.dy=self.dy*factor
            self.dz=self.dz*factor
        if coarse:
            self.dx=self.dx*(1/factor); self.dy=self.dy*(1/factor); self.
↪dz=self.dz*(1/factor) #Gjør mesh grovere

    def rho2(self, x, y, z):
        """
        Funksjon for varierende massetetthet
        Args:
            x, y, z - Koordinater til punkt
        Returnerer:
            Massetetthet i punkt
        """
        points = self.points
        dpoints = self.dpoints
        for i in range(points.shape[0]):#Looper over punktene og sjekke om de␣
↪er like
            point = points[i]
```

```
            #Toleranser
            if (np.abs(point[0] - x) < 1e-5 and
                np.abs(point[1] - y) < 1e-5 and
                np.abs(point[2] - z) < 1e-5):
                    return dpoints[i]
        d = np.zeros(points.shape[0], dtype=np.float64)
        for k in range(points.shape[0]): #Looper over punktene
            dx = x - points[k, 0]  #X avstand
            dy = y - points[k, 1]  #Y avstand
            dz = z - points[k, 2]  #Z avstand
            d[k] = (dx**2 + dy**2 + dz**2)**0.5 #Avstand
            if d[k] < 1e-5: #Toleranse for avstand
                    return dpoints[k]
        w = np.zeros(d.shape[0]) #Array for vekter
        for i in range(d.shape[0]): #Regner ut vekter
                w[i] = 1 / (d[i]**2)
        w_sum = np.sum(w) #Summen av vektene
        rho = np.sum(dpoints * w) / w_sum #Regner ut massetetthet
        if self.constant ==True:
            return 2400
        return rho


    def I(self, x, y, z):
        """
        Vektorisert nitialbetingelse for trykk
        Args:
            x, y, z (Arrays)
        Returnerer:
            Nuller i samme shape som mesh
        """
        return np.zeros_like(x * y * z)

    def qL(self, x, y, z):
        """
        Funksjon for varierende bølgehastighet"""

        rho=self.rho2
        K = self.K
        qL = K / rho(x, y, z)
        if self.constant:
            return self.c**2
        return qL

    def change_speed(self, c): #Gir konstant bølgefart
        constant = True
        self.c = c
```

```
        self.constant = constant

    def f(self, x, y, z, t, dt, wF=1500*2*np.pi):
        F = self.F
        A = (0.75 - 0.25) * (0.3 - 0.1) ##Areal for kraft
        f = -1 / (A) * F * wF**2* np.cos(wF * t)  #Kraft i punkt
        return np.where((0.25 <= x) & (x <= 0.75) & (0.1 <= y) & (y <= 0.3) &␣
↪(0.4 <= z) & (z <= 0.6), f, 0) ##Område hvor kraft virker

    def boundary_conditions_vectorized(self, x):
        #Looper over hver akse og setter randbetingelser
        d = np.zeros(x.shape, dtype=x.dtype)
        for i in range(1, x.shape[0] - 1):
            for j in range(1, x.shape[1] - 1):
                for k in range(1, x.shape[2] - 1):
                    d[i, j, k] = x[i, j, k]
        return d

    def advance_vectorized(self, p, p_n, p_nm1, xv, yv, zv, Cx2, Cy2, Cz2, dt,␣
↪t, n, qL_values, rho_values, step1=False):
        qL = self.qL
        f = self.f
        b= 1e+6 #Dempingskoeffisient
        #Hjelpevariabler
        rho_a = rho_values[1:-1, 1:-1, 1:-1]
        qL_a = qL_values[1:-1, 1:-1, 1:-1]
        qL_ip1 = qL_values[2:, 1:-1, 1:-1]
        qL_im1 = qL_values[:-2, 1:-1, 1:-1]
        qL_jp1 = qL_values[1:-1, 2:, 1:-1]
        qL_jm1 = qL_values[1:-1, :-2, 1:-1]
        qL_kp1 = qL_values[1:-1, 1:-1, 2:]
        qL_km1 = qL_values[1:-1, 1:-1, :-2]
        pq_ip1 =(qL_ip1+ qL_a) * (p_n[2:, 1:-1, 1:-1] - p_n[1:-1, 1:-1, 1:-1])
        pq_im1 = (qL_im1+qL_a) * (p_n[1:-1, 1:-1, 1:-1] - p_n[:-2, 1:-1, 1:-1])
        pq_jp1 = (qL_jp1+qL_a) * (p_n[1:-1, 2:, 1:-1] - p_n[1:-1, 1:-1, 1:-1])
        pq_jm1 =(qL_jm1+qL_a) * (p_n[1:-1, 1:-1, 1:-1] - p_n[1:-1, :-2, 1:-1])
        pq_kp1 = (qL_kp1+qL_a) * (p_n[1:-1, 1:-1, 2:] - p_n[1:-1, 1:-1, 1:-1])
        pq_km1 = (qL_km1+qL_a) * (p_n[1:-1, 1:-1, 1:-1] - p_n[1:-1, 1:-1, :-2])
        #Kraft for alle punkter
        f_values= f(xv, yv, zv, t[n], dt)
        f_values= f_values[1:-1, 1:-1, 1:-1]  #Kraft for indre punkter
        if step1: #Spesiell likning for første tidsteg
            p[1:-1, 1:-1, 1:-1] = (rho_a)**(-1)*0.5*(rho_a*2*p_n[1:-1, 1:-1, 1:
↪-1]+Cx2*0.5* (pq_ip1 - pq_im1)+Cy2 *0.5 *(pq_jp1 -pq_jm1) + Cz2 * 0.5 *␣
↪(pq_kp1 - pq_km1) * dt**2 * f_values)
        else:
```

```
            p[1:-1, 1:-1, 1:-1] = (rho_a+0.5*b*dt)**(-1)*((0.
↪5*b*dt-rho_a)*p_nm1[1:-1, 1:-1, 1:-1] + rho_a*2*p_n[1:-1, 1:-1, 1:-1] + Cx2␣
↪* 0.5 * (pq_ip1 - pq_im1) + Cy2 * 0.5 * (pq_jp1 - pq_jm1) + Cz2 * 0.5 *␣
↪(pq_kp1 - pq_km1))+dt**2 * f_values
        return p

    def adv_solver2dvectorized(self, x0=0, y0=0, z0=0):
        K=self.K; I = self.I; f = self.f; qL = self.qL
        dx = self.dx; dy = self.dy; dz = self.dz
        Nx = self.Nx; Ny = self.Ny; Nz = self.Nz; Nt = self.Nt
        xn = self.xn; yn = self.yn; zn = self.zn; tn = self.tn
        rho2 = self.rho2
        advance_vectorized = self.advance_vectorized
        boundary_conditions_vectorized = self.boundary_conditions_vectorized
        #Arrays
        x = np.linspace(x0, xn, Nx+1); dx =x[1]- x[0]
        y = np.linspace(y0, yn, Ny+1); dy = y[1]-y[0]
        z = np.linspace(z0, zn, Nz+1); dz =z[1]- z[0]
        #Vektoriserer x, y og z akser
        xv = x.reshape((x.size, 1, 1))
        yv = y.reshape((1, y.size, 1))
        zv = z.reshape((1, 1, z.size))
        #Oppretter arrays for trykk
        p = np.zeros((Nx+1,Ny +1, Nz+1))
        p_n = np.zeros((Nx+1, Ny+1, Nz +1))
        p_nm1 = np.zeros((Nx +1, Ny+1, Nz+1))

        rho_values = np.zeros((xv.shape[0], yv.shape[1], zv.shape[2]), dtype=np.
↪float64) ##Array for massetetthet
        #Loop over mesh og regner ut massetetthet i hvert punkt
        for i in range(len(x)):
            for j in range(len(y)):
                    for k in range(len(z)):
                        rho_values[i, j] = rho2(x[i], y[j], z[k])

        qL_values = K*1/(rho_values.copy()) #Bølgefart i hvert punkt (denne␣
↪linjen gjør qL-funksjonen unyttig)
        #Det gjør også at vi ikke kan sette konstant bølgefart i funksjonen qL,␣
↪men det er ikke nødvendig for 3D tilfellet uansett

        #Feilsjekker
        if np.max(qL_values)<= 0 or np.isnan(np.max(qL_values)):
            raise ValueError("Feil i qL_values | <=0 elr NaN")
        dx_dy_dz = np.array([dx, dy, dz], dtype=np.float64)
        if np.min(dx_dy_dz)<= 0 or np.isnan(np.min(dx_dy_dz)):
            raise ValueError("Feil i dx, dy, dz | <=0 elr NaN")
```

```
        B = 0.4 #Sikkerhetsfaktor
        dt = B*np.min(dx_dy_dz)/ np.sqrt(np.max(qL_values)) ␣
↪##Stabilitetsbetingelse

        if dt<= 0 or np.isnan(dt):
            raise ValueError("Feil i dt | <=0 elr NaN")

        #Lager tidspunkter og dt
        Nt = int(round(tn / dt))
        t = np.linspace(0, tn, Nt + 1)
        dt = t[1] - t[0]
        print(dt)

        Ix = range(0, Nx + 1); Iy = range(0, Ny + 1); Iz = range(0, Nz + 1); It␣
↪= range(0, Nt + 1) #Index-set notation (ikke i bruk)

        #Courant tall
        Cx2 = (dt/dx)**2
        Cy2 = (dt/dy)**2
        Cz2 = (dt/dz)**2

        p_data = []
        #Initialbetingelse
        p[:, :, :] = I(xv, yv, zv)
        p_data.append(p.copy())
        p_nm1, p_n, p = p_n, p, p_nm1

        #Første tidsssteg
        n = 0
        p = advance_vectorized(p, p_n, p_nm1, xv, yv, zv, Cx2, Cy2, Cz2, dt, t,␣
↪n, qL_values, rho_values, step1=True)
        p = boundary_conditions_vectorized(p)
        p_data.append(p.copy())
        p_nm1, p_n, p = p_n, p, p_nm1

        #Loop over tid
        for n in range(1, Nt + 1):
            p = advance_vectorized(p, p_n, p_nm1, xv, yv, zv, Cx2, Cy2, Cz2,␣
↪dt, t, n, qL_values, rho_values, step1=False)
            p = boundary_conditions_vectorized(p)
            p_data.append(p.copy())
            p_nm1, p_n, p = p_n, p, p_nm1
        return p_data, x, y, z, t

def breaking3D(p_data, tensilelim, compresslim, t):
        """
```

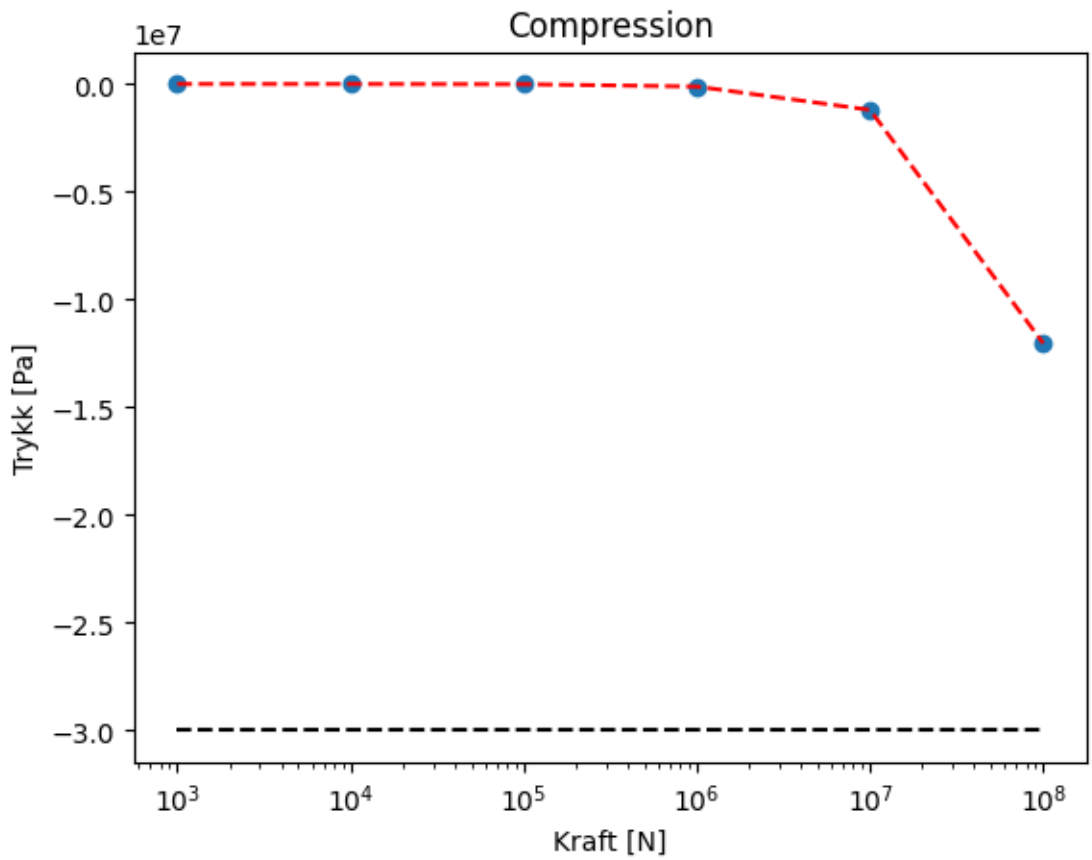
```
    Deler opp i tre deler og sjekker om trykkverdier er over eller under␣
↪grenseverdier
    Args:
        p_data (list) - Liste med trykkverdier for hver tid.
        tensilelim (float) - Øvre grenseverdi.
        compresslim (float) - Nedre grenseverdi.
        t (array) - Tidspunkter.

    Returnerer:
        results (bool) - True hvis trykkverdier er innenfor grenseverdier,␣
↪False ellers.
        vals2 (list) - Liste med min og max trykkverdier i de tre delene.
    """
    p_data = np.array(p_data)
    dt = t[1]-t[0]
    Nx, Ny, Nz = p_data[0].shape #Drar ut lengde på akser
    width = Nx // 3 ## Bredde for hver del


    part1 = p_data[:, :, :width, :] # Venstre del
    part2 = p_data[:,  :, width:2 * width, :]  #Midtre del
    part3 = p_data[:, :, 2 * width:, :] # Høyre del

    tvals = np.zeros(3)
    parts = [part1, part2, part3]

    for i, part in enumerate(parts, start=0): #Looper over hver del
        last_satisfied_time = -10 #Initaliserer til -10
        for n in range(p_data.shape[0], ):  # Loop over tid
            if np.any(np.logical_or(part[n] > tensilelim, part[n] <␣
↪compresslim)): #Betingelser
                last_satisfied_time = n*dt #Tidspunkt
                break
        tvals[i] = last_satisfied_time
    print(tvals)
    #Returnerer True hvis grensebetingelser er oppfylt
    if np.any(tvals<0): #Hvis noen av delene er mindre enn null (altså␣
↪initialisert verdi -10) -> sprekk ikke formet
        results=False
    else:
        results=True

    least = [np.min(part) for part in parts] #Finner minste punkt i hver del
    biggest = [np.max(part) for part in parts] #Finner største punkt i hver␣
↪del
    minval = np.max(least) #Finner minste av de største
    maxval = np.min(biggest) #Finner største av de minste
```

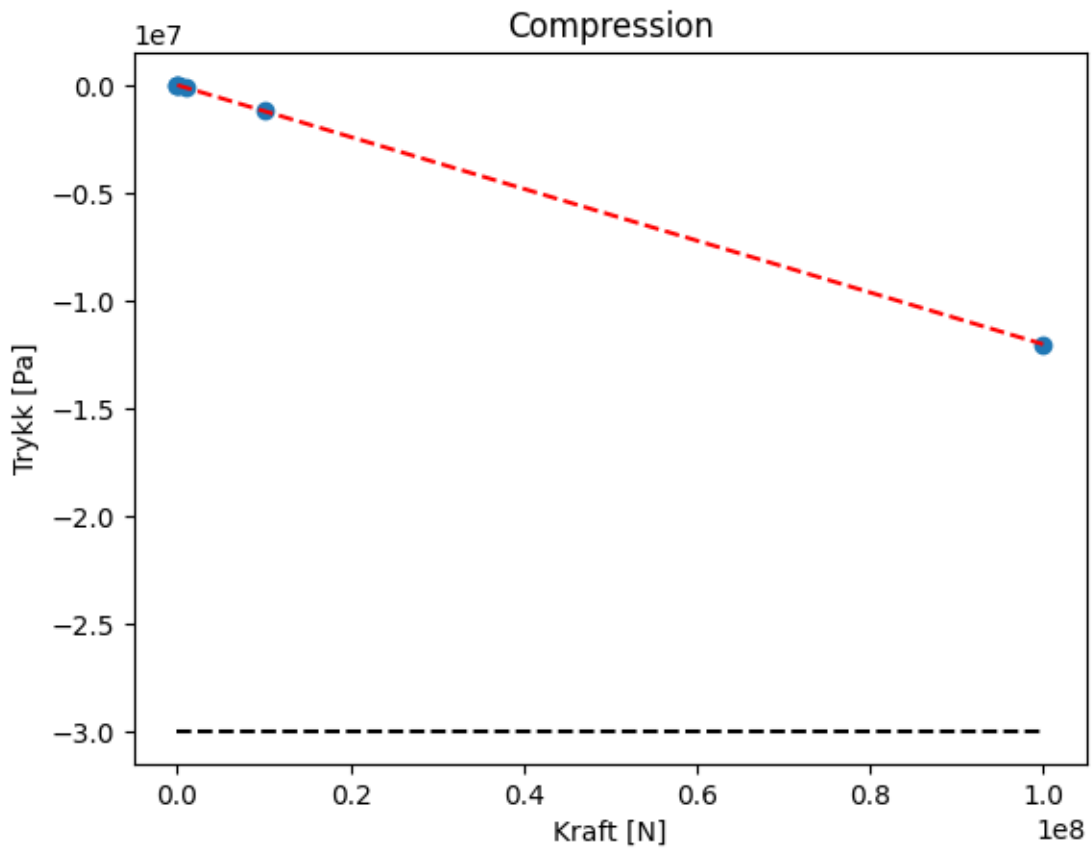
```
        vals2 = [minval, maxval]

        return results, vals2
def animate3D(p_data, x, y, z, t, save=False, file=None, skipframes=1):
    """
    Funksjon for å animere 3D trykkverdier
    Args:
        p_data (list) - Liste med trykkverdier for hver tid.
        x, y, z (array) - Mesh i x, y og z retning.
        t (array) - Tidspunkter.
        save (bool) - Om animasjonen skal lagres som gif.
        file (str) - Filnavn for gif.
        skipframes (int) - Antall bilder som skal hoppes over.
    """
    fig = plt.figure(figsize=(8, 8))
    ax = plt.axes(projection='3d')

    pmin = np.min(p_data)
    umax = np.max(p_data)
    X, Y, Z = np.meshgrid(x, y, z)

    #Lager en ScalarMappable for colourbar og farger
    norm = plt.Normalize(vmin=pmin, vmax=umax)
    sm = plt.cm.ScalarMappable(cmap="magma", norm=norm)
    sm.set_array([])

    def update(frame): #Oppdaterer animasjonen
        ax.clear()
        X_masked = X[np.abs(p_data[frame]) > 0.1 * umax]
        Y_masked = Y[np.abs(p_data[frame]) > 0.1 * umax]
        Z_masked= Z[np.abs(p_data[frame]) > 0.1 * umax]
        p_data = p_data[frame][np.abs(p_data[frame]) > 0.1 * umax]


        normalized_colors = norm(p_data) #Normaliserer fargene

        #Plotting
        ax.set_title(f"Time: {t[frame-1]:.3f}")
        ax.set_xlabel("X")
        ax.set_ylabel("Y")
        ax.set_zlabel("Z")
        ax.set_xlim((0, 1))
        ax.set_ylim((0, 1))
        ax.set_zlim((0, 1))
        scatter= ax.scatter(X_masked, Y_masked, Z_masked, c=normalized_colors,␣
↪cmap="magma", norm=norm, alpha=0.9, marker="o", edgecolors="face")
        return scatter
```

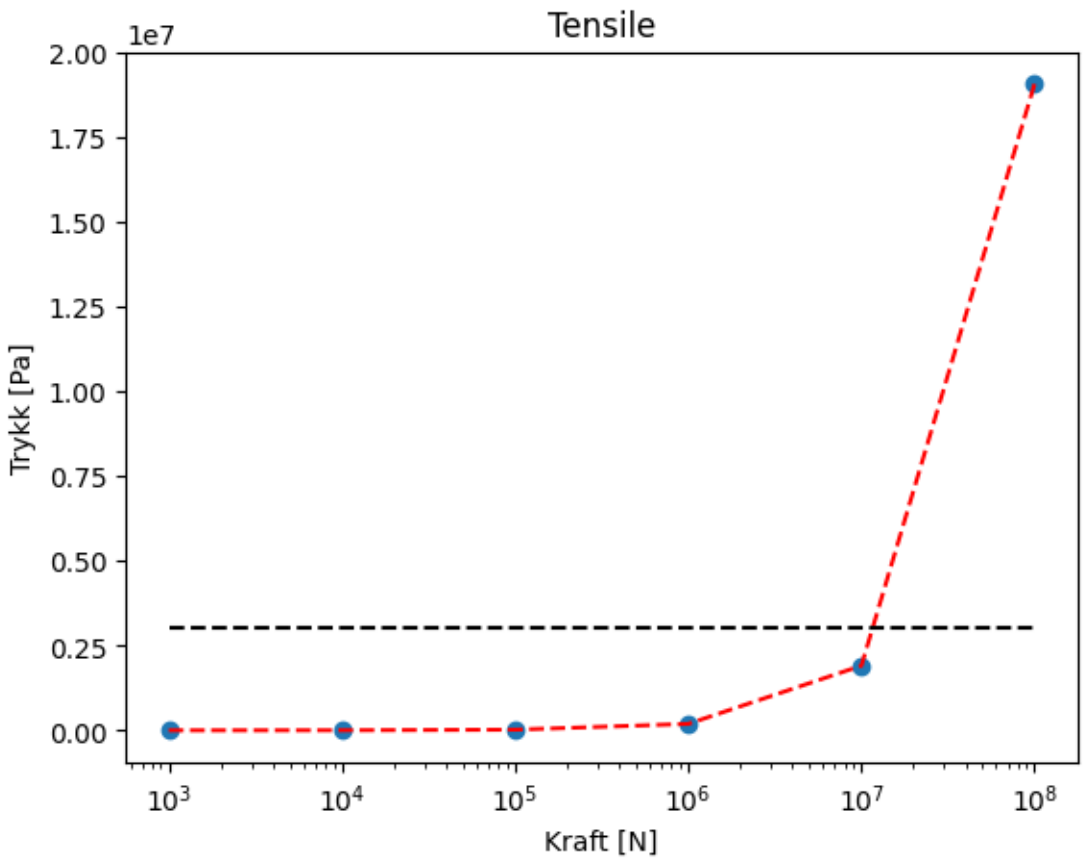
```
    # Colourbar
    cbar = fig.colorbar(sm, ax=ax, shrink=0.5, aspect=10)
    cbar.set_label("Amplitude")

    #Lagring og visning av animasjon
    if save==True:
        save_count=len(p_data)
        writer="PillowWriter"
    else:
        save_count, writer = None, None
    anim = FuncAnimation(fig,update,frames=range(0, len(p_data),␣
↪skipframes),interval=100, save_count=save_count)
    if save:
        anim.save(f"{file}.gif", writer=writer)
    plt.show()



f = [10**i for i in range(3, 10)]
compresslim = -30e+6; tensilelim=3e+6

def foo3(b, f, compresslim, tensilelim, constant=False):
    """
    Funksjon for å kjøre wavesolver og sjekke om grensebetingelser er oppfylt
    Args:
        b (float) - Dempingskoeffisient
        f (list) - Liste med kraftmodulus
        compresslim (float) - Nedre grenseverdi
        tensilelim (float) - Øvre grenseverdi
        constant (bool) - Om massetetthet og bølgefart skal være konstant
    Returnerer:
        least (list) - Liste med trykkverdi nærmest compressbetingelse
        biggest (list) - Liste med trykkverdi nærmest tensilebetingelse
        fnew (list) - Liste med kraftmodulus
    """
    #Arrayer for å lagre verdier
    fnew = []
    least = []
    biggest = []
    for i in range(len(f)):
        c = wavesolver3D(b, F=f[i], dx=1, dy=1, dz=1, Nx=5, Ny=5, Nz=5, Nt=5,␣
↪xn=1, yn=1, zn=1, tn=2) #Ny instans av wavesolver
        if constant: #Setter konstant fart og massetetthet hvis ønskelig
            c.change_speed(3000, True)
        p, x, y, z, t = c.adv_solver2dvectorized() #Løser bølgelikning
```

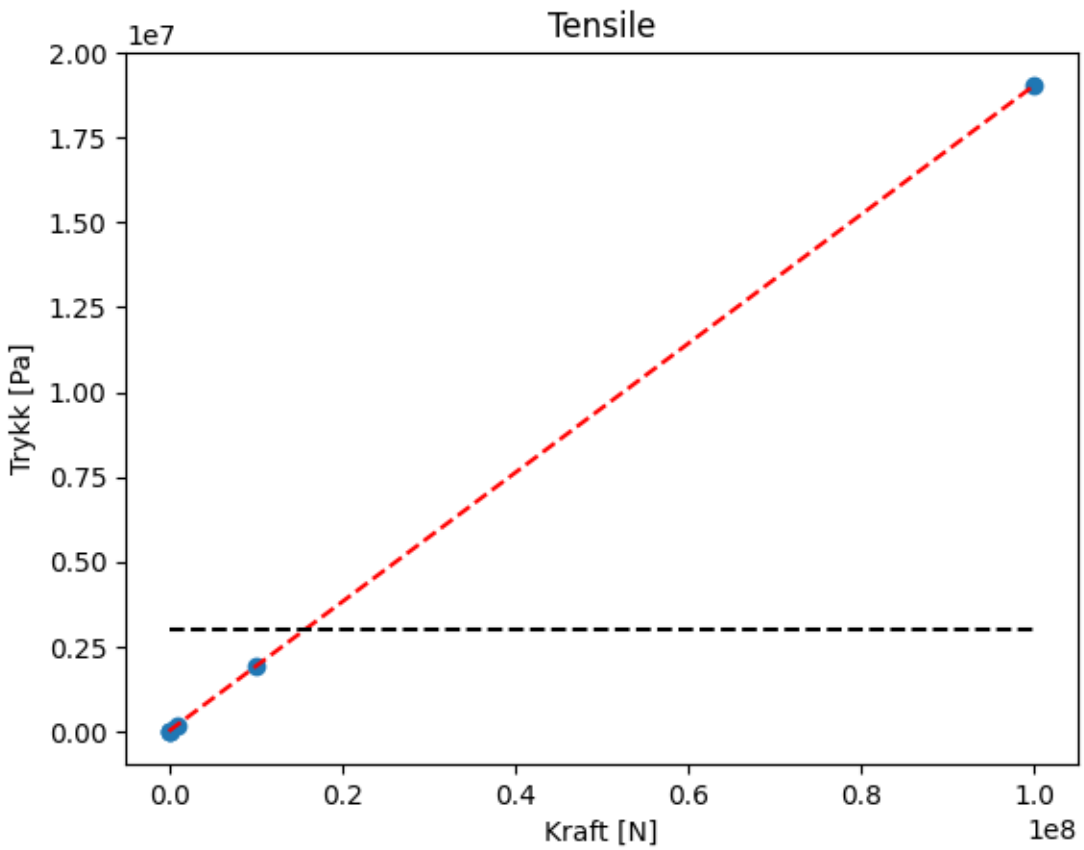
```
        results, vals = breaking3D(p, tensilelim, compresslim, t) #Sjekker␣
↪grensebetingelser
        results = results #True hvis grensebetingelser er oppfylt
        least_element = vals[0] #Nærmest compressbetingelse
        biggest_element = vals[1] #Nærmest tensilebetingelse
        biggest.append(biggest_element)
        least.append(least_element)
        fnew.append(f[i])
        if results == True:
            break
    return least, biggest, fnew

least, biggest, fnew = foo3(1e+6, f, compresslim, tensilelim, constant=False)
plotting(least, fnew, compresslim, "Compression")
plotting(biggest, fnew, tensilelim, "Tensile")
```

```
2.294604238134028e-05
[-10. -10. -10.]
2.294604238134028e-05
[-10. -10. -10.]
2.294604238134028e-05
[-10. -10. -10.]
2.294604238134028e-05
[ 5.7365106e-04 -1.0000000e+01 -1.0000000e+01]
2.294604238134028e-05
[ 1.14730212e-04  2.34049632e-03 -1.00000000e+01]
2.294604238134028e-05
[4.58920848e-05 1.17024816e-03 5.62178038e-03]
```


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
10^3
10^4
10^5
10^6
10^7
10^8
Kraft [N]


Compression
1e7
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Trykk [Pa]
0.0
0.2
0.4
0.6
0.8
1.0
1e8
Kraft [N]


Tensile
1e7
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Trykk [Pa]
10^3
10^4
10^5
10^6
10^7
10^8
Kraft [N]



I 3D tilfelle har kraften økt med en størrelsesørden. Kraften er nå omtrent på størresle 10^7 Newton. En av årsakene til dette er at 2D systemet vårt ikke helt oversetter til 3D systemet. Selv om vi har samme verdier for dempingskoeffisient, masse, osv., så har vi valgt et vilkårlig z-interval for kraftområdet: $x \in [0.25, 0.75], y \in [0.1, 0.3] z \in [0.4, 0.6]$. Vi observerer likevel samme lineær trend som i det udempede og dempede 2D-tilfellet.

11 Konklusjon

I denne rapporten har vi sett at The Flash trenger å utøve en kraft på omtrent 10^7 Newton for å skape sprekker i betongen.

Forholdet mellom kraft og formasjonen av sprekker er lineært. Vi observerer tidspunkter for sprekkene som er forventet, spesielt med tanke på dempingsfaktoren. Det tar mindre tid for sprekkene å formere seg i dempede 3D tilfellet, men tiden er innenfor samme størrelsesorden for dempet 2D. For å forbedre nøyaktigheten er en mulighet å kjøre kodene med finere resolusjon i meshen, selv om det gjør beregningene mer krevende.

Resultatene gjelder derimot bare under bestemte antagelser: Betongen er tilnærmet som en væske, slik at bare longitudinale bølger er modelert.

Grensebetingelsene vi har satt tilsvarer en blokk som "svever" i atmosfærisk trykk. I virkeligheten

er betongveggen knyttet til forskjellige materialer. Det representerer et generelt problem med bølgemodellering, som er at det er ekstremt vanskelig å finne perfekte grensebetingelser for et realistisk system.

I tillegg har vi ikke tatt hensyn til hvordan formasjonene av sprekkene kontinuerlig endrer betongens egenskaper.

12 Referanser

[1] Mydatabook "Bulk Modulus" Mydatabook. https://www.mydatabook.org/solid-mechanics/bulk-modulus/ (Hentet 20.04.2025)

[2] L. C. Herazo. "Bulk density, bulk porosity, compressive strength, Young's modulus, bending strength" Researchgate. https://www.researchgate.net/figure/Bulk-density-bulk-porosity-compressive-strength-Youngs-modulus-bending-strength_tbl4_348383256 (hentet 21.04.2025)

[3] Pile Dyanmics. "Is there a relation between concrete strength and wave speed?" Pile. "https://www.pile.com/resources/faqs/is-there-a-relation-between-concrete-strength-and-wave-speed-how-can-i-determine-the-minimum-acceptable-wave-speed/" (Hentet 19.04.2025)

[4] Pile Dynamics. "How is the real wave velocity found by a Pile Integrity Test (PIT)?" Pile. https://www.pile.com/resources/faqs/faq-find-real-wave-velocity-pile-integrity-test/#:~:text=The%20wave%20speed%20in%20concrete,high%20as%204400%20m%2Fsec. (Hentet 19.04.2025)

[5] A. I. Vistnes. "Physics of Oscillations and Waves Arnt Inge Vistnes With use of Matlab and Python" Switzerland, Springer, 2016.

[6] H.P Langtagen, S Linge. "Finite Difference Computing with PDEs A Modern Software Approach" Switzerland, Cham, Springer Open, 2017

[7] C Heinemeyer "EUROPEAN DESIGN GUIDE FOR FOOTBRIDGE VIBRATION" https://www.researchgate.net/figure/Damping-ratios-x-according-to-construction-material-11-12_tbl2_266506141 (Hentet 21.04.2025)

[8] Wikipedia. "Concrete" https://en.wikipedia.org/wiki/Concrete (Hentet 21.04.2025)

[9] ReadyMixConcrete "What Is The Standard Strength Of Concrete?" https://www.bigdreadymix.com/what-is-the-standard-strength-of-concrete/#:~:text=Studies%20indicate%20that%20traditional%20concrete's,10%25%20of%20the%20compressive% (Hentet 21.04.2025) [10] Wikipedia. https://no.wikipedia.org/wiki/General_Electric_GE90. Wikipedia (Hentet 21.04.2025)